UNIVERSIDADE PAULISTA – UNIP
Projeto Integrado Multidisciplinar
Curso Superior de Tecnologia em
Análise e Desenvolvimento de Sistemas

NOME DO ALUNO – RA

PATAFÁCIL: SISTEMA INTEGRADO DE GESTÃO PARA PET SHOP E CLÍNICA VETERINÁRIA

Desenvolvimento e implantação de uma solução tecnológica corporativa integrada para apoiar a transformação digital

CIDADE
2026

NOME DO ALUNO – RA

PATAFÁCIL: SISTEMA INTEGRADO DE GESTÃO PARA PET SHOP E CLÍNICA VETERINÁRIA

Desenvolvimento e implantação de uma solução tecnológica corporativa integrada para apoiar a transformação digital

Projeto Integrado Multidisciplinar em Análise e Desenvolvimento de Sistemas

Projeto Integrado Multidisciplinar para obtenção do título de tecnólogo em Análise e Desenvolvimento de Sistemas, apresentado à Universidade Paulista – UNIP.

Orientador(a): NOME DO(A) ORIENTADOR(A)

CIDADE

2026

RESUMO

Este trabalho apresenta o desenvolvimento do PataFácil, uma solução tecnológica corporativa integrada para apoiar a transformação digital de um pet shop com clínica veterinária de pequeno porte, dando continuidade ao estudo realizado no PIM III. O objetivo foi construir e validar um sistema que unifica agenda de serviços, prontuário clínico, estoque, vendas e indicadores gerenciais, acessível por aplicação web e por aplicativo móvel. A pesquisa é aplicada, de abordagem qualitativa, apoiada em levantamento bibliográfico e conduzida de forma iterativa e incremental com o framework Scrum. A solução adota arquitetura em camadas e orientada a objetos: uma API REST em ASP.NET Core concentra as regras de negócio e é consumida tanto pela aplicação web, em HTML, CSS e JavaScript, quanto pelo aplicativo em React Native; os dados ficam em um banco MySQL com triggers, procedures, views e índices; e um microsserviço em Python fornece previsões de risco de cancelamento, de demanda e de segmentação de clientes. A autenticação utiliza tokens JWT, com senhas protegidas por BCrypt e autorização por perfil. O projeto contempla ainda o plano de infraestrutura em nuvem com containers e pipeline de integração e entrega contínuas, além de ações de acessibilidade, diversidade e responsabilidade social. No balcão, o sistema registra a chegada do cliente e o pagamento, emite automaticamente a ordem de serviço em A4, PDF ou cupom para impressora térmica e controla o recebimento de mercadorias com nota fiscal, validade, margem de lucro e cadastro de fornecedores. A validação reuniu 28 testes unitários automatizados, verificações de ponta a ponta sobre a API e testes diretos dos objetos do banco, todos aprovados. Conclui-se que a solução atende aos objetivos propostos, elimina os conflitos de agenda, integra as vendas ao estoque e oferece ao gestor informações para decidir com base em dados.

Palavras-chave: Transformação digital. Sistema integrado. API REST. Aplicativo móvel. Banco de dados. Metodologias ágeis.

ABSTRACT

This work presents the development of PataFácil, an integrated corporate technology solution designed to support the digital transformation of a small pet shop with a veterinary clinic, continuing the study carried out in PIM III. The objective was to build and validate a system that unifies service scheduling, clinical records, inventory, sales and management indicators, available through a web application and a mobile application. This is applied research with a qualitative approach, supported by a literature review and conducted iteratively and incrementally with the Scrum framework. The solution adopts a layered, object-oriented architecture: a REST API in ASP.NET Core concentrates the business rules and is consumed both by the web application, written in HTML, CSS and JavaScript, and by the React Native application; data is stored in a MySQL database with triggers, stored procedures, views and indexes; and a Python microservice provides predictions of cancellation risk, demand and customer segmentation. Authentication uses JWT tokens, with passwords protected by BCrypt and role-based authorization. The project also includes a cloud infrastructure plan with containers and a continuous integration and delivery pipeline, as well as accessibility, diversity and social responsibility actions. At the front desk, the system records the customer's arrival and payment, automatically issues the service order as an A4 page, a PDF file or a thermal printer receipt, and manages the receipt of goods with invoice, expiry date, profit margin and supplier registration. Validation comprised 28 automated unit tests, end-to-end checks on the API and direct tests of the database objects, all of which passed. It is concluded that the solution meets the proposed objectives, eliminates scheduling conflicts, integrates sales with inventory and gives the manager information to make data-driven decisions.

Keywords: Digital transformation. Integrated system. REST API. Mobile application. Database. Agile methodologies.

LISTA DE FIGURAS

Figura 1 – Organograma da organização 10
Figura 2 – Arquitetura em camadas da solução 17
Figura 3 – Visão geral da equipe, com indicadores, gráficos do mês e previsão de demanda 20
Figura 4 – Agenda do balcão com um atendimento em processo 21
Figura 5 – Lista de ordens de serviço da semana 21
Figura 6 – Ordem de serviço no formato A4, pronta para impressão ou PDF 22
Figura 7 – Entrada de mercadoria com alertas de reposição e de validade 23
Figura 8 – Sequência de integração na criação de um agendamento 24
Figura 9 – Telas de login, início, pets e consultas do aplicativo 25
Figura 10 – Passos do assistente de agendamento no aplicativo 26
Figura 11 – Fluxo de navegação do aplicativo 26
Figura 12 – Diagrama de classes do domínio 28
Figura 13 – Modelo entidade-relacionamento do banco de dados 32
Figura 14 – Arquitetura de referência em nuvem 36
Figura 15 – Pipeline de integração e entrega contínuas 37
Figura 16 – Tela de login e cadastro 56
Figura 17 – Portal do tutor 56
Figura 18 – Assistente de agendamento da aplicação web 57
Figura 19 – Consultório veterinário e prontuário eletrônico 57
Figura 20 – Venda no balcão com carrinho e alertas de estoque 58
Figura 21 – Painel do gerente com o relatório de faturamento 58
Figura 22 – Caixa de e-mail do usuário 59
Figura 23 – Ordem de serviço no formato de cupom para impressora térmica de 80 mm 59

LISTA DE QUADROS

Quadro 1 – Serviços e produtos oferecidos pela organização 9
Quadro 2 – Benefícios esperados e recursos que os viabilizam 13
Quadro 3 – Modelo de negócio do PataFácil (Business Model Canvas) 14
Quadro 4 – Principais endpoints da API REST 18
Quadro 5 – Endpoints consumidos pelo aplicativo 27
Quadro 6 – Camadas da solução e suas responsabilidades 29
Quadro 7 – Modelo lógico: relações, atributos principais e chaves 32
Quadro 8 – Stored procedures implementadas 34
Quadro 9 – Triggers implementadas 34
Quadro 10 – Resultados dos testes executados no banco de dados 35
Quadro 11 – Serviços de nuvem previstos 37
Quadro 12 – Product Backlog 39
Quadro 13 – Cronograma das Sprints e Sprint Backlog resumido 41
Quadro 14 – Quadro Kanban na primeira semana da Sprint 4 41
Quadro 15 – Entregas por Sprint 42

SUMÁRIO

1 INTRODUÇÃO 8
2 CARACTERIZAÇÃO DA ORGANIZAÇÃO 9
2.1 Segmento de atuação 9
2.2 Produtos e serviços 9
2.3 Estrutura organizacional 10
2.4 Principais processos 10
2.5 Problemas identificados 10
2.6 Oportunidades de melhoria por meio da tecnologia 11
3 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA 12
3.1 Problema a ser resolvido 12
3.2 Objetivos da solução 12
3.3 Público-alvo 13
3.4 Proposta de valor 13
3.5 Benefícios esperados 13
3.6 Diferenciais competitivos 13
3.7 Modelo de negócio e viabilidade 14
4 RESPONSABILIDADE SOCIAL E DIVERSIDADE 15
4.1 Combate à discriminação 15
4.2 Valorização da diversidade 15
4.3 Recursos tecnológicos inclusivos 15
4.4 Ações voltadas à acessibilidade 16
4.5 Utilização da tecnologia para promoção da cidadania 16
5 DESENVOLVIMENTO DA SOLUÇÃO WEB 17
5.1 Arquitetura da aplicação 17
5.2 APIs REST 18
5.3 Autenticação 18
5.4 Segurança 19
5.5 Estrutura administrativa 19
5.6 Integração entre módulos 23
6 DESENVOLVIMENTO DA SOLUÇÃO MOBILE 25
6.1 Telas principais 25
6.2 Fluxo de navegação 26

6.3 Autenticação 27
6.4 Sincronização de dados 27
6.5 Integração com APIs 27
7 ARQUITETURA DE SOFTWARE 28
7.1 Orientação a objetos 28
7.2 Organização em camadas 29
7.3 Modularização 29
8 PROJETO DO BANCO DE DADOS 31
8.1 Modelo entidade-relacionamento 31
8.2 Modelo lógico 32
8.3 Modelo físico 33
8.4 Procedures 33
8.5 Triggers 34
8.6 Script completo e validação 34
9 INFRAESTRUTURA EM NUVEM E DEVOPS 36
9.1 Arquitetura em nuvem 36
9.2 Serviços utilizados 36
9.3 Containers 37
9.4 Pipeline de CI/CD 37
9.5 Monitoramento 38
9.6 Escalabilidade 38
9.7 Segurança 38
10 GERENCIAMENTO ÁGIL DO PROJETO 39
10.1 Papéis e eventos 39
10.2 Product Backlog 39
10.3 Sprint Backlog e cronograma das Sprints 40
10.4 Quadro Kanban 41
10.5 Definição das entregas 41
11 CONCLUSÃO 43
REFERÊNCIAS 44
APÊNDICE A – SCRIPT COMPLETO DO BANCO DE DADOS 47
APÊNDICE B – TELAS COMPLEMENTARES DA APLICAÇÃO WEB 56

1 INTRODUÇÃO

A transformação digital tornou-se um dos principais fatores de competitividade das organizações. Empresas de todos os portes buscam integrar processos, automatizar atividades, ampliar a disponibilidade dos serviços e oferecer melhor experiência a seus clientes. Nos pequenos negócios esse movimento ainda encontra rotinas manuais e informações dispersas, que geram retrabalho, perda de receita e decisões tomadas sem dados.

Este Projeto Integrado Multidisciplinar (PIM IV) dá continuidade ao PIM III, no qual foram levantados os requisitos e modelada a solução para a PataFácil, um pet shop com clínica veterinária de pequeno porte adotado como estudo de caso. A empresa presta serviços de banho e tosa, consultas e vacinação, além de vender produtos para animais de estimação, e até então controlava a agenda, os prontuários e o estoque em papel e planilhas.

O objetivo geral deste trabalho é desenvolver e validar uma solução tecnológica corporativa integrada para a organização, contemplando o planejamento do negócio, as aplicações web e mobile, as APIs de integração, a arquitetura de software, o banco de dados, o plano de infraestrutura em nuvem com práticas de DevOps e o gerenciamento ágil do projeto, sem perder de vista a inclusão, a acessibilidade e a responsabilidade social.

Quanto à metodologia, trata-se de uma pesquisa aplicada, de abordagem qualitativa e caráter exploratório, apoiada em pesquisa bibliográfica (GIL, 2017; MARCONI; LAKATOS, 2017; SEVERINO, 2016) sobre engenharia de software, bancos de dados, computação em nuvem e métodos ágeis. A construção do sistema seguiu um ciclo iterativo e incremental orientado pelo Scrum (SCHWABER; SUTHERLAND, 2020), e cada incremento foi verificado por testes automatizados e por testes de integração executados sobre a aplicação em funcionamento.

O trabalho está organizado conforme as etapas do manual do PIM IV. Os capítulos 2 a 4 tratam da organização, do planejamento da solução e da responsabilidade social. Os capítulos 5 a 8 descrevem a aplicação web, o aplicativo mobile, a arquitetura de software e o banco de dados. Os capítulos 9 e 10 apresentam a infraestrutura em nuvem com DevOps e o gerenciamento ágil. Por fim, o capítulo 11 traz a conclusão, seguida das referências e dos apêndices.

2 CARACTERIZAÇÃO DA ORGANIZAÇÃO

A PataFácil é um estabelecimento de cuidados com animais de estimação que reúne, no mesmo endereço, um pet shop e uma clínica veterinária. Atende tutores de cães, gatos, aves e pequenos roedores em uma região urbana residencial e opera em uma única loja, de segunda a sábado, das 8h às 19h. Por se tratar de um estudo de caso acadêmico, os dados aqui apresentados descrevem o cenário analisado no PIM III.

2.1 Segmento de atuação

A organização atua no setor de comércio e serviços para animais de estimação, combinando o varejo de produtos com a prestação de serviços estéticos e veterinários. Enquadra-se como microempresa e concorre com outros estabelecimentos de bairro e com redes especializadas, frente às quais a agilidade e a proximidade no atendimento são seus principais atributos.

2.2 Produtos e serviços

O Quadro 1 resume o catálogo cadastrado no sistema: 25 serviços, com a duração padrão usada para reservar a agenda do profissional, e 51 produtos vendidos no balcão, distribuídos em cinco categorias.

Quadro 1 – Serviços e produtos oferecidos pela organização

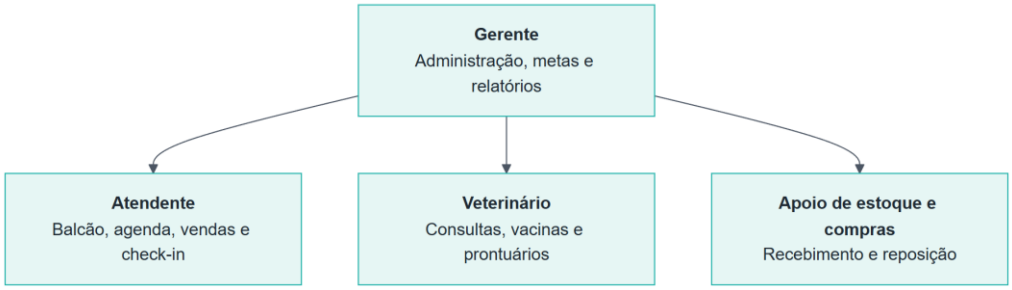
Tipo	Item	Descrição
Serviço	Banho e tosa (12 itens)	Banho por porte, tosa na máquina, na tesoura, higiênica e bebê, hidratação, banho medicamentoso e banho para gatos; de 40 a 90 minutos
Serviço	Consultas (4 itens)	Consulta, retorno, emergência e check-up geriátrico; de 20 a 60 minutos
Serviço	Vacinas e procedimentos (9 itens)	Vacinas V10, antirrábica, V4 felina e gripe canina, vermifugação, microchipagem, hemograma e limpeza de tártaro; de 15 a 90 minutos
Produto	Alimentos	Rações secas e úmidas, petiscos e alimentos para aves e roedores
Produto	Medicamentos	Antipulgas, vermífugos, suplementos e pomadas
Produto	Higiene	Shampoos, tapetes higiênicos, areias sanitárias e itens de limpeza
Produto	Acessórios e outros	Coleiras, guias, camas, comedouros, brinquedos, caixas de transporte e placas de identificação

Fonte: elaborado pelo autor (2026).

2.3 Estrutura organizacional

A estrutura é funcional e enxuta, como mostra a Figura 1. O gerente responde pela administração, pelas metas e pela equipe; o atendente cuida do balcão, da agenda e das vendas; o veterinário realiza os atendimentos clínicos; e o apoio de estoque recebe e repõe as mercadorias. Esses papéis deram origem aos perfis de acesso do sistema.

Figura 1 – Organograma da organização



Fonte: elaborado pelo autor (2026).

2.4 Principais processos

Os processos que sustentam a operação, e que orientaram o escopo da solução, são os seguintes:

a) agendamento de serviços: o tutor solicita o serviço e a equipe encaixa o horário na agenda do profissional;
b) atendimento clínico: o veterinário atende o animal e registra a consulta no prontuário;
c) atendimento no balcão: o atendente recebe o cliente, acompanha o serviço e recebe o pagamento ao final;
d) venda no balcão: o atendente registra a venda de produtos e recebe o pagamento;
e) reposição de estoque: o responsável confere os saldos, solicita compras aos fornecedores e confere a nota fiscal e a validade no recebimento;
f) fechamento financeiro: o gerente consolida as receitas de serviços e de produtos.

2.5 Problemas identificados

O levantamento apontou que a operação manual gerava falhas recorrentes. A agenda em papel permitia marcar dois serviços para o mesmo profissional no

mesmo horário; as faltas dos clientes não eram previstas e deixavam horários ociosos; os produtos acabavam sem aviso, pois a venda não baixava o estoque; o histórico clínico ficava disperso em fichas; o tutor dependia do telefone para agendar; e o gerente não dispunha de indicadores confiáveis para decidir.

2.6 Oportunidades de melhoria por meio da tecnologia

Cada problema corresponde a uma oportunidade de melhoria: uma agenda que impeça conflitos automaticamente; canais digitais, web e mobile, que deem autonomia ao tutor; prontuário eletrônico com lembrete de vacinas; estoque integrado às vendas, com alerta de reposição; modelos de inteligência artificial para estimar a demanda e o risco de cancelamento; e relatórios gerenciais em tempo real. Essas oportunidades fundamentam o planejamento descrito no próximo capítulo.

3 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA

Dornelas (2018) destaca que empreender é transformar uma oportunidade identificada em um negócio viável, o que exige definir com clareza o problema, o público e o valor entregue. Este capítulo apresenta esse planejamento para o PataFácil.

3.1 Problema a ser resolvido

O problema central é a ausência de uma solução integrada que automatize o fluxo operacional do estabelecimento, composto por agenda, clínica, estoque e vendas, e que ao mesmo tempo aproxime o cliente por canais digitais. A fragmentação das informações causa retrabalho, conflitos de horário, ruptura de estoque e decisões sem base em dados.

3.2 Objetivos da solução

O objetivo da solução é digitalizar e integrar os processos da organização em uma única plataforma. Especificamente, pretende-se:

a) impedir conflitos de horário para o mesmo profissional (regra de negócio RN01);
b) permitir que o tutor agende, acompanhe e cancele serviços on-line, respeitando a antecedência mínima de duas horas (RN02) e o horário de funcionamento (RN03);
c) controlar o ciclo do agendamento em etapas travadas, de pendente a confirmado e a concluído (RN04), com a duração calculada pelo sistema (RN10);
d) manter o histórico clínico de cada animal em prontuário eletrônico;
e) integrar vendas e estoque, com baixa automática e alerta no ponto de reposição (RN15);
f) registrar o recebimento de mercadorias com nota fiscal, validade, margem de lucro e fornecedor;
g) emitir a ordem de serviço de cada atendimento, para impressão comum, arquivo PDF ou impressora térmica;
h) oferecer relatórios e indicadores ao gerente e previsões por aprendizado de máquina.

3.3 Público-alvo

A solução atende dois públicos. O primeiro são os tutores, clientes do estabelecimento, em geral moradores da região, usuários de smartphone e que valorizam praticidade. O segundo é a equipe interna, formada por atendentes, veterinários e gerente, com níveis distintos de familiaridade com tecnologia, o que motivou interfaces simples e específicas para cada perfil.

3.4 Proposta de valor

A proposta de valor pode ser resumida em reunir todos os cuidados com o animal em um só lugar. Para o tutor, significa agendar a qualquer hora, escolher profissional e horário entre opções realmente disponíveis e consultar o histórico do animal. Para o estabelecimento, significa uma agenda sem conflitos, estoque sob controle e informação para decidir.

3.5 Benefícios esperados

O Quadro 2 relaciona os benefícios esperados ao recurso do sistema que os viabiliza.

Quadro 2 – Benefícios esperados e recursos que os viabilizam

Benefício	Como a solução entrega
Fim dos conflitos de agenda	Validação de sobreposição de horários por profissional (RN01)
Redução de faltas	Previsão do risco de cancelamento a cada novo agendamento
Menos ruptura de estoque	Baixa automática na venda e alerta no ponto de reposição (RN15)
Melhor experiência do cliente	Portal web e aplicativo para agendar, acompanhar e cancelar
Decisão baseada em dados	Indicadores do dia, relatório de faturamento e previsão de demanda
Prontuário organizado	Histórico clínico digital por animal, com data da próxima vacina
Comprovante para o cliente	Ordem de serviço gerada automaticamente, em A4, PDF ou cupom térmico
Compras sob controle	Entrada de mercadoria com nota, validade, margem sugerida e fornecedor cadastrado
Segurança da informação	Autenticação JWT, senhas com BCrypt e acesso por perfil

Fonte: elaborado pelo autor (2026).

3.6 Diferenciais competitivos

Em relação às soluções usuais para pequenos estabelecimentos, o PataFácil se diferencia por quatro aspectos: o uso de inteligência artificial para

antecipar comportamentos, e não apenas registrar dados; a oferta dos canais web e mobile sobre as mesmas APIs, o que garante consistência; a aplicação das regras de negócio no servidor e no banco de dados, e não só na interface; e o baixo custo, por empregar apenas tecnologias de código aberto ou gratuitas.

3.7 Modelo de negócio e viabilidade

O modelo de negócio foi estruturado com o Business Model Canvas (OSTERWALDER; PIGNEUR, 2011) e está sintetizado no Quadro 3. A solução é oferecida como serviço, por assinatura mensal, o que dispensa o cliente de manter infraestrutura própria.

Quadro 3 – Modelo de negócio do PataFácil (Business Model Canvas)

Bloco	Definição
Segmentos de clientes	Pet shops e clínicas veterinárias de pequeno e médio porte e seus tutores
Proposta de valor	Gestão integrada, conveniência digital para o tutor e previsões por IA
Canais	Aplicação web, aplicativo mobile e notificações por e-mail
Relacionamento	Autoatendimento no portal e no aplicativo, com suporte ao estabelecimento
Fontes de receita	Assinatura mensal por estabelecimento e módulos adicionais
Recursos principais	Equipe de desenvolvimento, infraestrutura em nuvem e base de dados
Atividades principais	Desenvolvimento, operação da plataforma e suporte
Parcerias principais	Provedor de nuvem e serviço de e-mail transacional
Estrutura de custos	Hospedagem, manutenção evolutiva e atendimento

Fonte: elaborado pelo autor (2026).

A análise de viabilidade é favorável nas três dimensões consideradas. Tecnicamente, as tecnologias adotadas são consolidadas e bem documentadas. Operacionalmente, o sistema automatiza rotinas que a equipe já executa, o que reduz a curva de aprendizado. Financeiramente, não há custo de licenciamento e a hospedagem em nuvem é paga conforme o uso, de modo que o investimento se concentra no desenvolvimento e tende a ser compensado pela redução de faltas e de perdas de estoque.

4 RESPONSABILIDADE SOCIAL E DIVERSIDADE

A tecnologia não é neutra: as escolhas de projeto podem ampliar ou reduzir desigualdades. Almeida (2019) mostra que o racismo opera de modo estrutural, reproduzido por práticas institucionais mesmo sem intenção individual, e Munanga e Gomes (2006) reforçam que superar essa condição exige ações concretas de valorização da diversidade. Com base nessa compreensão e na legislação brasileira, a solução incorporou as ações descritas a seguir.

4.1 Combate à discriminação

O sistema trata todos os usuários de forma igual: serviços, preços e horários são os mesmos para qualquer tutor, e o cadastro não coleta raça, cor, religião ou renda, em linha com o princípio da não discriminação do Estatuto da Igualdade Racial (BRASIL, 2010) e com a minimização de dados prevista na Lei Geral de Proteção de Dados (BRASIL, 2018). Os modelos de inteligência artificial utilizam apenas variáveis comportamentais, como antecedência e histórico de cancelamentos, o que evita que uma decisão automatizada reproduza preconceitos.

4.2 Valorização da diversidade

As mensagens e telas adotam linguagem respeitosa e neutra, e o acesso aos módulos depende apenas da função exercida. Propõe-se à organização que a identidade visual e os materiais de divulgação representem a diversidade étnico-racial da população brasileira e que o processo de contratação valorize essa diversidade, em sintonia com as diretrizes de educação das relações étnico-raciais instituídas a partir da Lei nº 10.639 (BRASIL, 2003).

4.3 Recursos tecnológicos inclusivos

Todas as páginas da aplicação web incorporam o VLibras, conjunto de ferramentas gratuitas do governo federal que traduz conteúdos digitais para a Língua Brasileira de Sinais (BRASIL, 2026). Os formulários de acesso possuem rótulos descritivos para leitores de tela, e o aplicativo utiliza componentes nativos compatíveis com os recursos de acessibilidade do Android e do iOS. O agendamento em passos curtos reduz a carga cognitiva e favorece pessoas idosas ou com pouca familiaridade com tecnologia.

4.4 Ações voltadas à acessibilidade

As ações de acessibilidade seguem a Lei Brasileira de Inclusão (BRASIL, 2015) e as diretrizes WCAG 2.1 (W3C, 2018), organizadas em quatro frentes: comunicação, com a tradução em Libras; percepção visual, com contraste entre texto e fundo e hierarquia tipográfica clara; navegação, com fluxos curtos e botões amplos; e conteúdo, com textos diretos e sem jargões. Como evolução, recomenda-se realizar testes com usuários com deficiência e ampliar os atributos de acessibilidade nas telas internas.

4.5 Utilização da tecnologia para promoção da cidadania

Ao permitir que o tutor acompanhe seus agendamentos e o histórico do animal, a solução amplia o acesso à informação e a transparência na relação de consumo. A oferta simultânea de web e aplicativo favorece a inclusão digital; o prontuário com lembrete de vacinas estimula a guarda responsável e o bem-estar animal; o controle de estoque reduz desperdícios; e o modelo de baixo custo torna ferramentas de gestão acessíveis a pequenos empreendedores.

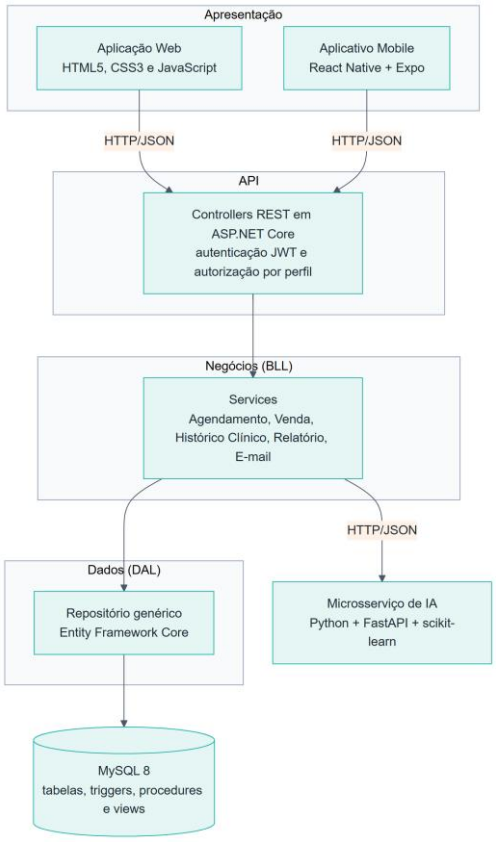
5 DESENVOLVIMENTO DA SOLUÇÃO WEB

A aplicação web é composta por um servidor de APIs em ASP.NET Core (MICROSOFT, 2026a) e por páginas em HTML5, CSS3 e JavaScript que consomem essas APIs pelo navegador.

5.1 Arquitetura da aplicação

A solução segue a arquitetura em camadas, que separa a apresentação, as regras de negócio e o acesso a dados para que cada parte evolua sem afetar as demais (SOMMERVILLE, 2018). A Figura 2 apresenta as camadas e o sentido das chamadas: as interfaces enviam requisições HTTP aos controllers, que acionam os serviços de negócio, e estes usam os repositórios para ler e gravar no MySQL. O microsserviço de inteligência artificial é acionado pela camada de negócio.

Figura 2 – Arquitetura em camadas da solução



Fonte: elaborado pelo autor (2026).

5.2 APIs REST

As APIs adotam o estilo arquitetural REST (FIELDING, 2000): os recursos são identificados por URLs, manipulados pelos verbos HTTP e representados em JSON. O Quadro 4 lista os principais endpoints e os perfis autorizados em cada um.

Quadro 4 – Principais endpoints da API REST

Método e rota	Finalidade	Perfis
POST /api/auth/login	Autentica e devolve o token JWT	Público
POST /api/tutores	Cadastra um tutor	Público
GET e POST /api/pets	Lista e cadastra os animais do tutor	Tutor e equipe
GET /api/servicos	Catálogo de serviços ativos	Público
GET /api/agendamento/horarios-disponiveis	Horários livres por data, serviço e profissional	Público
POST /api/agendamento	Cria o agendamento (RN01, RN03 e RN10)	Tutor e atendente
DELETE /api/agendamento/{id}	Cancela o agendamento (RN02)	Tutor, atendente e gerente
PATCH /api/agendamento/{id}/confirmar e /concluir	Avança o status (RN04)	Equipe
PATCH /api/agendamento/{id}/chegada e /pagamento	Registra a chegada do cliente e a baixa do pagamento	Atendente e gerente
GET /api/agendamento/periodo e /em-atendimento	Agenda do balcão e ordens de serviço por período	Equipe
GET /api/agendamento/tutor/{id}	Agendamentos do tutor	Tutor (os próprios) e balcão
POST e GET /api/historicoclinico	Registra e consulta o prontuário	Veterinário; tutor (leitura)
GET e POST /api/estoque/entradas	Recebimento de mercadoria com nota, validade e preço	Atendente e gerente
GET e POST /api/estoque/fornecedores	Lista e cadastra fornecedores	Atendente e gerente
GET /api/estoque/produtos e /critico	Produtos, saldos, validades e estoque crítico	Atendente e gerente
POST /api/vendas	Registra a venda com baixa de estoque (RN15)	Atendente e gerente
GET /api/relatorios/painel e /receita-mensal	Indicadores, gráficos do mês e faturamento	Equipe; gerente (financeiro)
GET, PATCH e DELETE /api/emails	Caixa de e-mail do usuário logado	Todos os perfis
GET /api/ml/previsao-demanda-hoje	Previsão de demanda obtida da IA	Equipe
GET /health	Verificação de saúde da API e do banco	Público

Fonte: elaborado pelo autor (2026).

5.3 Autenticação

A autenticação utiliza JSON Web Token, padrão que transporta declarações assinadas entre as partes (JONES; BRADLEY; SAKIMURA, 2015). O usuário informa e-mail e senha; a API confere a senha com o hash armazenado e emite um token assinado com o algoritmo HMAC-SHA256, válido por oito horas, que contém o

identificador, o e-mail e o perfil do usuário. O cliente envia esse token no cabeçalho Authorization de cada requisição, e o servidor valida a assinatura e a validade sem manter sessão, o que favorece a escalabilidade.

5.4 Segurança

As medidas de segurança foram aplicadas em camadas complementares:

a) senhas armazenadas apenas como hash BCrypt de fator 12, inclusive as da equipe inicial;
b) autorização por perfil em cada endpoint: o relatório financeiro, por exemplo, responde com o código 403 a qualquer usuário que não seja o gerente;
c) verificação de propriedade: o tutor só acessa os próprios animais, agendamentos e históricos, mesmo que altere os identificadores da requisição;
d) validação de entrada no servidor, com rejeição de e-mail ou CPF duplicados, e respostas que nunca expõem o hash da senha;
e) mensagem única para e-mail inexistente e senha incorreta, para não revelar quais contas existem;
f) escape dos textos recebidos da API antes da exibição nas páginas, prevenindo a injeção de código.

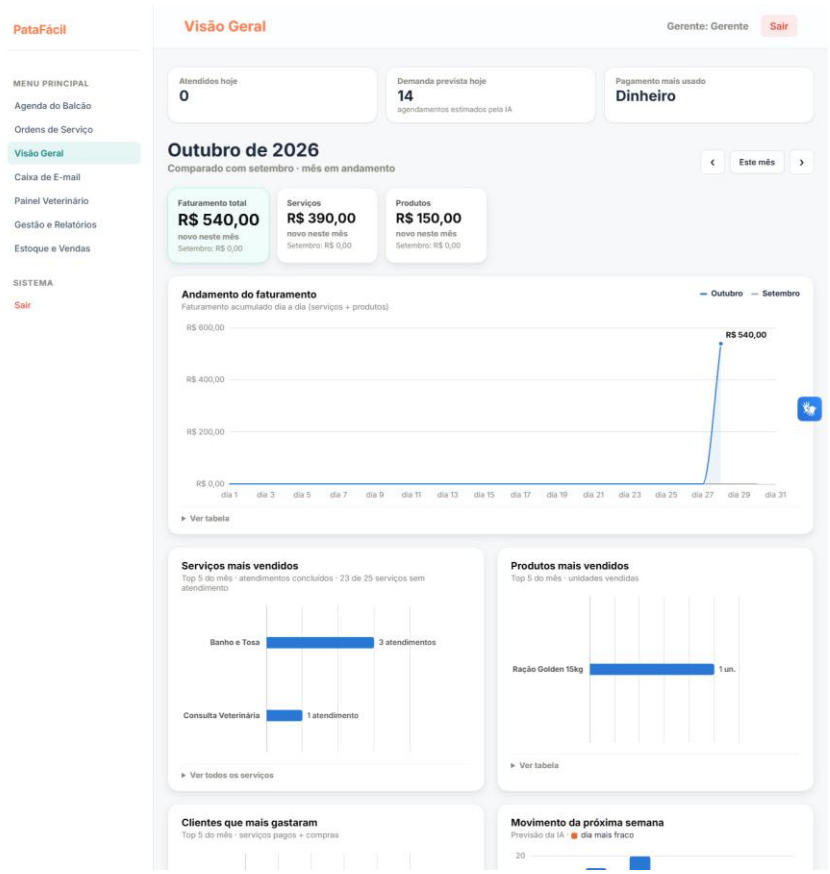
5.5 Estrutura administrativa

Após o login, cada perfil é direcionado ao seu painel e o menu lateral mostra apenas os módulos permitidos. O tutor acessa o portal, com seus animais e agendamentos, o assistente de agendamento em passos e a caixa de e-mail. A interface segue um padrão visual único em todas as telas, com a fonte Inter, própria para leitura em tela, cores sólidas e poucos elementos decorativos; os ícones ilustrados ficam restritos aos avatares dos animais.

A visão geral, mostrada na Figura 3, reúne os indicadores do dia, a demanda prevista pela inteligência artificial e o desempenho do mês comparado ao anterior: faturamento acumulado dia a dia, os cinco serviços e os cinco produtos mais vendidos, os clientes que mais gastaram, o movimento previsto para a semana e sugestões de promoção geradas a partir da validade dos lotes, do estoque parado e

do movimento previsto. Cada gráfico tem uma tabela equivalente, para quem prefere ler os números.

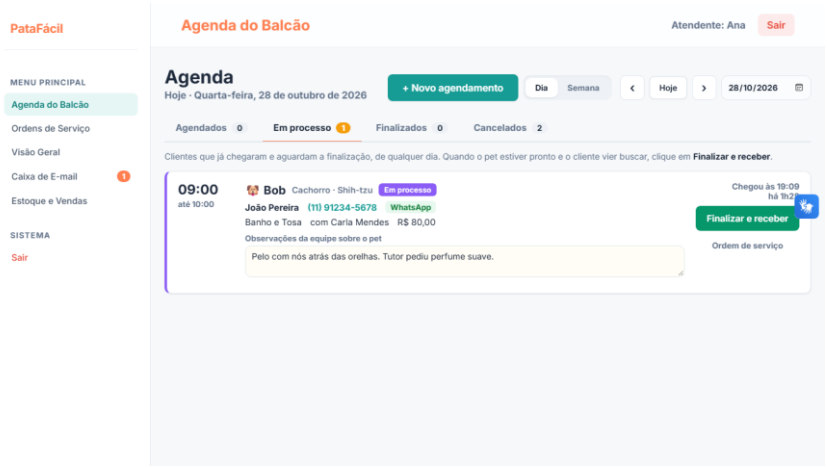
Figura 3 – Visão geral da equipe, com indicadores, gráficos do mês e previsão de demanda



Fonte: elaborado pelo autor (2026).

O atendente trabalha na agenda do balcão, apresentada na Figura 4. A agenda é navegada por dia ou por semana e organizada em abas que acompanham o atendimento: agendados, em processo, finalizados e cancelados. Cada cartão mostra o animal, o cliente com telefone e atalho para o WhatsApp, o serviço, o profissional e um campo de observações da equipe. O botão "Cliente chegou" leva o atendimento para a aba em processo, e "Finalizar e receber" registra o valor, a forma de pagamento e o troco. O balcão também agenda serviços para clientes que chegam sem cadastro prévio.

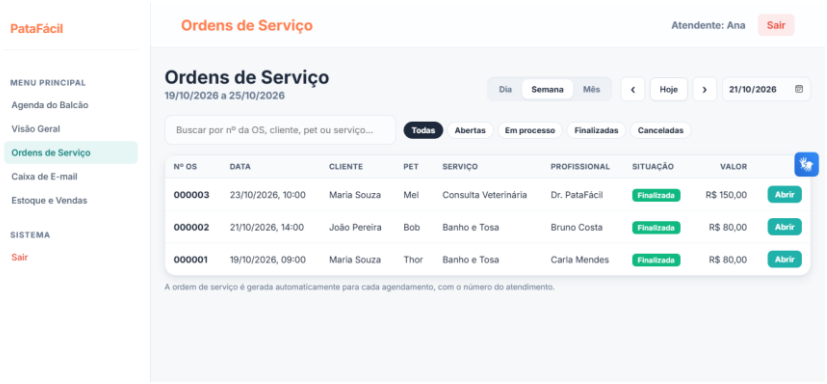
Figura 4 – Agenda do balcão com um atendimento em processo



Fonte: elaborado pelo autor (2026).

Para cada agendamento o sistema gera automaticamente uma ordem de serviço, numerada pelo próprio atendimento, que reúne os dados do cliente e do animal, o serviço, o profissional, as observações e o pagamento. A tela de ordens de serviço, na Figura 5, lista as ordens do dia, da semana ou do mês, com busca e filtro por situação.

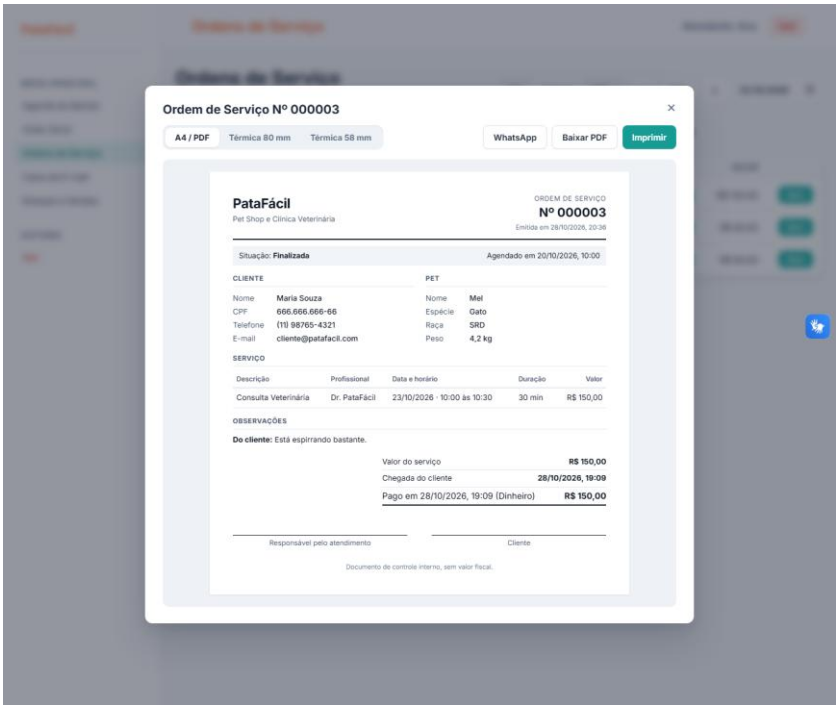
Figura 5 – Lista de ordens de serviço da semana



Fonte: elaborado pelo autor (2026).

Ao abrir uma ordem, escolhe-se o formato, como mostra a Figura 6: folha A4, que pode ser impressa ou baixada como arquivo PDF para envio ao cliente, ou cupom de 80 mm ou 58 mm para impressoras térmicas, cuja altura é calculada pelo conteúdo. Um atalho abre o WhatsApp do cliente com o resumo do atendimento.

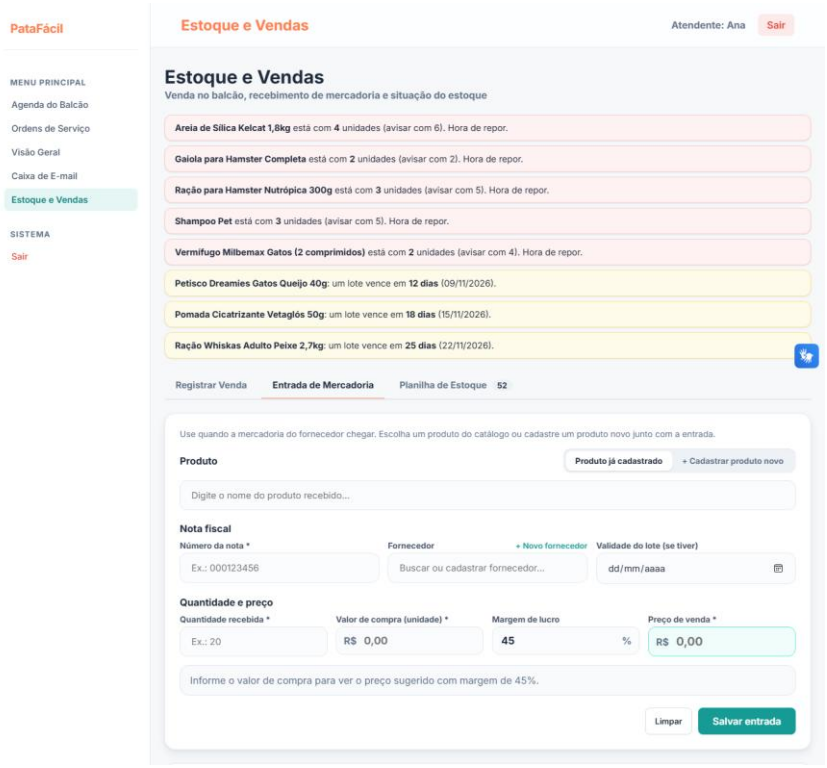
Figura 6 – Ordem de serviço no formato A4, pronta para impressão ou PDF



Fonte: elaborado pelo autor (2026).

Na tela de estoque e vendas, a aba de entrada de mercadoria, na Figura 7, registra o recebimento do fornecedor com número da nota, validade do lote, quantidade, custo e preço de venda. O sistema sugere o preço pela margem de lucro sobre a venda e mostra o lucro por unidade. O produto pode ser escolhido no catálogo ou cadastrado na própria entrada, e o fornecedor é escolhido entre os já cadastrados, com o último fornecedor do produto preenchido automaticamente, ou cadastrado na hora, com CNPJ, telefone e e-mail. Os alertas do topo indicam produtos no ponto de reposição e lotes que vencem em até 30 dias.

Figura 7 – Entrada de mercadoria com alertas de reposição e de validade



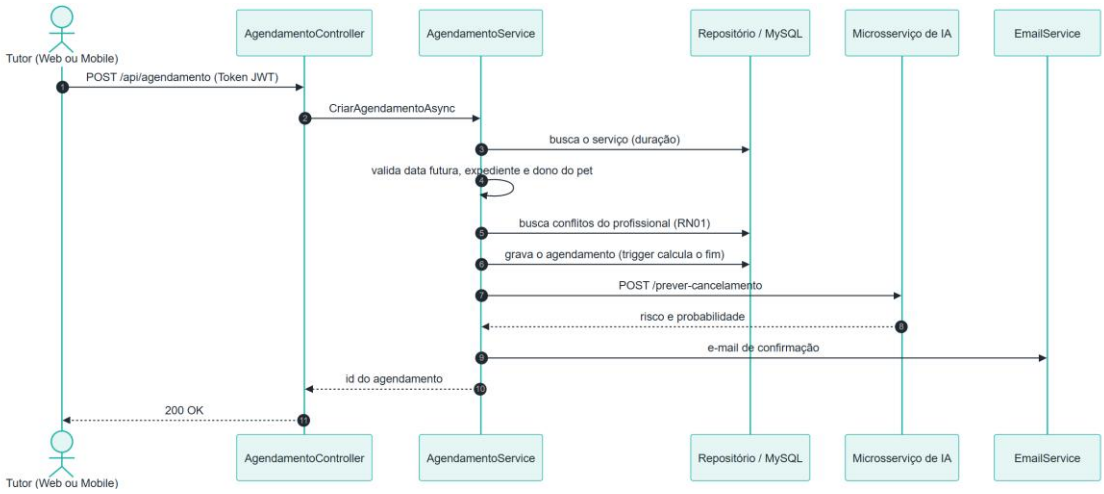
Fonte: elaborado pelo autor (2026).

O veterinário trabalha no consultório, onde vê a fila de pacientes e registra o prontuário, e o gerente dispõe ainda do relatório de faturamento e do cadastro da equipe. Todos os perfis têm uma caixa de e-mail com as mensagens que o sistema enviou, como confirmações e cancelamentos. As demais telas constam do Apêndice B.

5.6 Integração entre módulos

Os módulos trocam informações de forma automática. A Figura 8 detalha a criação de um agendamento: o serviço de negócio valida as regras, grava o registro, consulta o microsserviço de inteligência artificial para anotar o risco de cancelamento e dispara a notificação por e-mail, simulada no ambiente de desenvolvimento. A consulta à inteligência artificial tem tempo limite de três segundos e sua indisponibilidade não impede o agendamento.

Figura 8 – Sequência de integração na criação de um agendamento



Fonte: elaborado pelo autor (2026).

De modo análogo, o registro do prontuário conclui o agendamento confirmado na mesma operação, e cada item de venda aciona, no banco de dados, a baixa do estoque e a geração do alerta de reposição. A entrada de mercadoria atualiza o saldo, o custo e o preço do produto, registra o lote com a validade e, se o fornecedor informado ainda não existir, inclui-o no cadastro.

A validação dessa integração reuniu 28 testes unitários, escritos com xUnit e Moq, que cobrem as regras de agendamento (RN01, RN02, RN03, RN04 e RN10), a chegada e o pagamento no balcão, a posse dos dados pelo tutor, as vendas com baixa de estoque (RN15), a entrada de mercadoria com o cálculo do preço pela margem e o armazenamento de senhas com BCrypt. Os testes dispensam o banco de dados, pois os repositórios são simulados. Complementarmente, foram executadas verificações de ponta a ponta sobre a API em funcionamento, cobrindo as regras de negócio, as restrições de acesso e o cadastro de fornecedores; todos os testes foram aprovados.

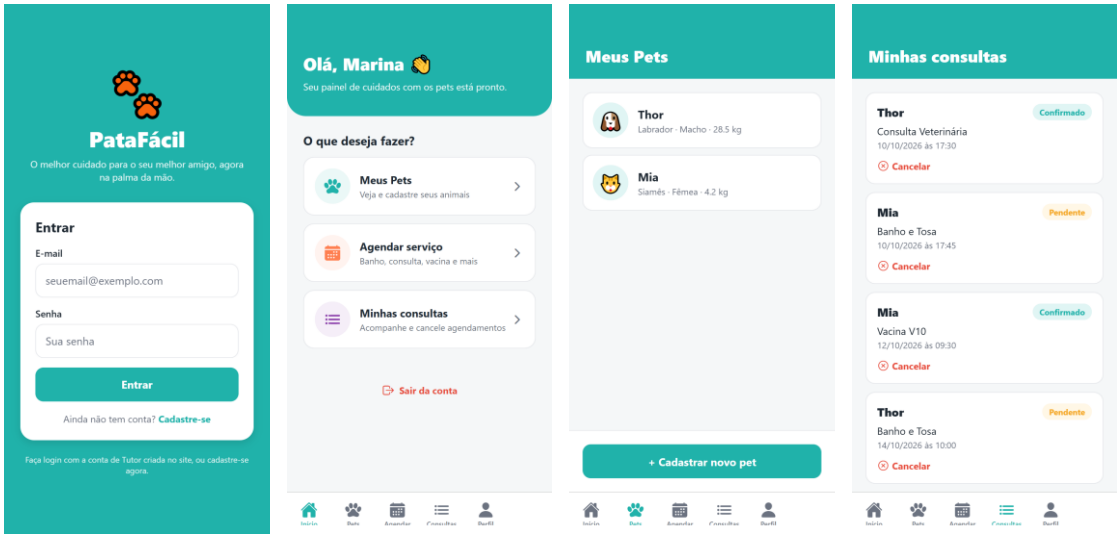
6 DESENVOLVIMENTO DA SOLUÇÃO MOBILE

O aplicativo móvel destina-se ao tutor e foi desenvolvido em React Native (META PLATFORMS, 2026) com a plataforma Expo (EXPO, 2026) e a linguagem TypeScript, o que permite gerar versões para Android e iOS a partir do mesmo código.

6.1 Telas principais

O aplicativo possui oito telas. As Figuras 9 e 10 reúnem as principais, capturadas com o aplicativo em execução e conectado à API. A tela inicial oferece atalhos para as funções mais usadas; a tela de animais lista os pets do tutor e dá acesso ao cadastro; a tela de consultas mostra os agendamentos com o respectivo status e a opção de cancelamento.

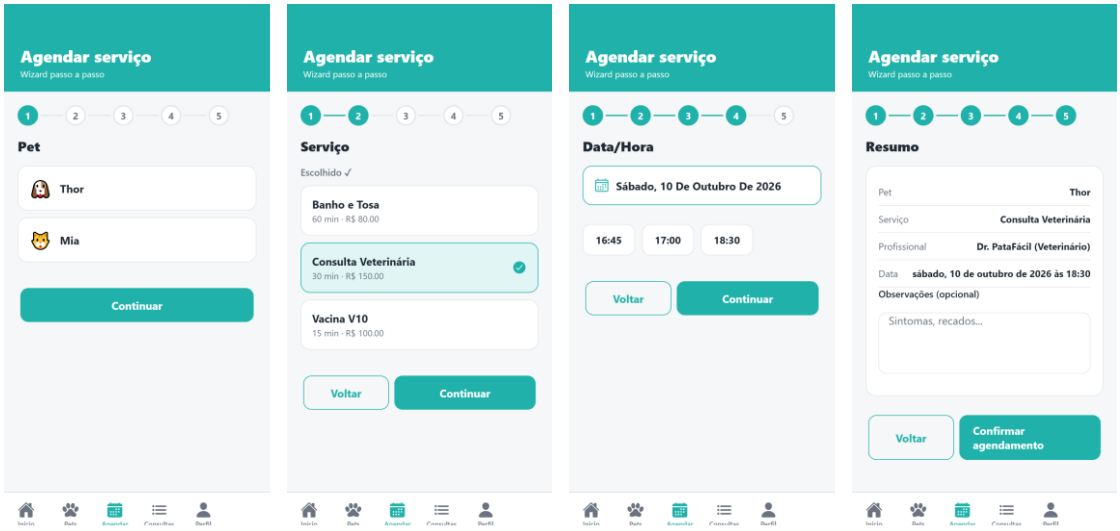
Figura 9 – Telas de login, início, pets e consultas do aplicativo



Fonte: elaborado pelo autor (2026).

O agendamento é conduzido por um assistente de cinco passos: escolha do animal, do serviço e do profissional, seleção da data e de um dos horários livres informados pela API e, por fim, o resumo para confirmação.

Figura 10 – Passos do assistente de agendamento no aplicativo

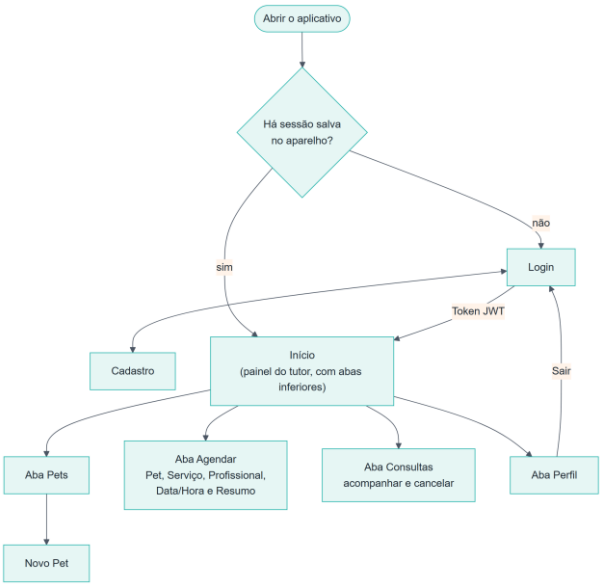


Fonte: elaborado pelo autor (2026).

6.2 Fluxo de navegação

A navegação combina uma pilha de telas para o acesso e abas inferiores para a área logada, conforme a Figura 11. Ao abrir o aplicativo, verifica-se se há sessão salva: em caso negativo, exibem-se as telas de login e de cadastro; em caso positivo, o usuário segue diretamente para o painel, cujas abas dão acesso a início, pets, agendar, consultas e perfil.

Figura 11 – Fluxo de navegação do aplicativo



Fonte: elaborado pelo autor (2026).

6.3 Autenticação

O aplicativo autentica-se no mesmo endpoint da aplicação web. O token JWT e os dados básicos do usuário são gravados no armazenamento local do aparelho, de modo que a sessão é restaurada nas aberturas seguintes. Ao sair, esses dados são removidos. Um contexto de autenticação disponibiliza o token a todas as telas, e o cliente HTTP o anexa às requisições protegidas.

6.4 Sincronização de dados

Os dados de negócio não são mantidos em cópia local: a fonte única é a API, o que evita divergências entre os canais. As listas são recarregadas sempre que a tela recebe o foco e quando o usuário puxa a lista para baixo; após criar ou cancelar um agendamento, a consulta é refeita. Assim, uma confirmação feita pelo atendente na web aparece no aplicativo na atualização seguinte. As requisições têm tempo limite de quinze segundos e mensagens de erro compreensíveis para falhas de conexão.

6.5 Integração com APIs

A integração foi isolada em uma camada de serviços, com um cliente HTTP comum que monta as requisições, injeta o token e traduz os erros retornados pelo servidor. O Quadro 5 relaciona as telas aos endpoints consumidos, os mesmos da aplicação web.

Quadro 5 – Endpoints consumidos pelo aplicativo

Tela	Endpoints
Login e cadastro	POST /api/auth/login; POST /api/tutores
Pets e novo pet	GET /api/pets/tutor/{id}; POST /api/pets
Agendar	GET /api/servicos; GET /api/funcionarios/profissionais; GET /api/agendamento/horarios-disponiveis; POST /api/agendamento
Consultas	GET /api/agendamento/tutor/{id}; DELETE /api/agendamento/{id}

Fonte: elaborado pelo autor (2026).

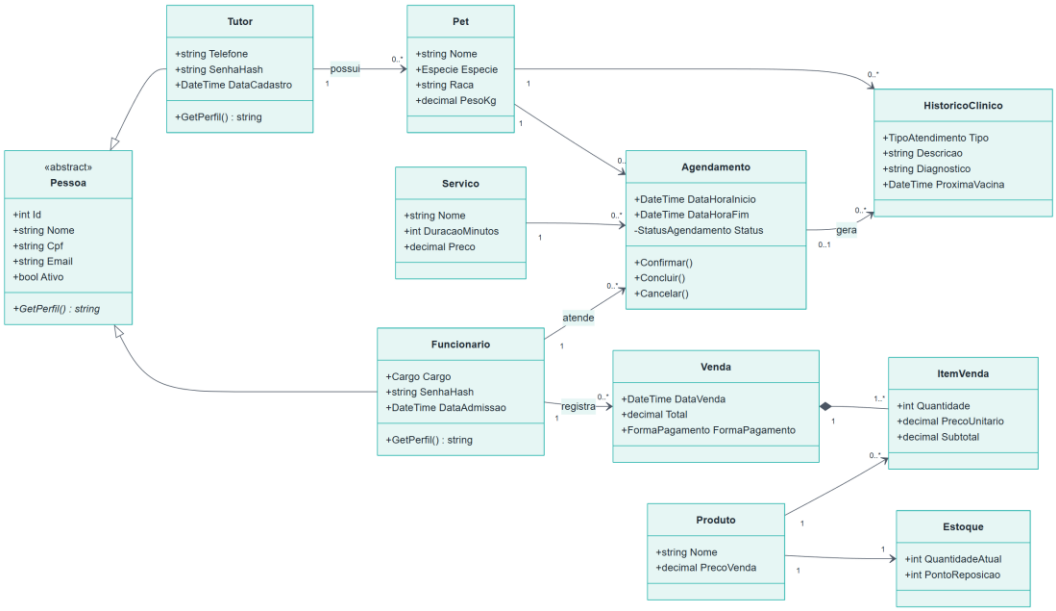
7 ARQUITETURA DE SOFTWARE

A arquitetura foi definida para atender simultaneamente à aplicação web e ao aplicativo, com um núcleo único de regras de negócio exposto por APIs.

7.1 Orientação a objetos

O domínio foi modelado com os pilares da orientação a objetos, representados no diagrama de classes da Figura 12. A classe abstrata Pessoa reúne os atributos comuns a tutores e funcionários (abstração), que são suas especializações (herança). O método GetPerfil é implementado de forma distinta em cada subclasse (polimorfismo), o que permite ao serviço de autenticação obter o perfil de qualquer usuário sem conhecer o seu tipo concreto.

Figura 12 – Diagrama de classes do domínio



Fonte: elaborado pelo autor (2026).

O encapsulamento protege as regras mais sensíveis. Na classe Agendamento, o status só pode ser alterado pelos métodos da própria classe, que verificam a transição permitida, como mostra o trecho a seguir; desse modo, a regra RN04 vale para qualquer ponto do sistema que manipule um agendamento.

```
public StatusAgendamento Status { get; private set; } =
StatusAgendamento.PENDENTE;

public void Concluir()
{
    // RN04: o fluxo é travado (Pendente -> Confirmado -> Concluído)
    if (Status != StatusAgendamento.CONFIRMADO)
        throw new InvalidOperationException(
            "Apenas agendamentos confirmados podem ser concluídos.");
    Status = StatusAgendamento.CONCLUIDO;
}
```

7.2 Organização em camadas

O Quadro 6 descreve as camadas e sua localização no código. Uma requisição percorre sempre o mesmo caminho, do controller ao serviço e deste ao repositório, sem que uma camada acesse diretamente outra que não seja a imediatamente inferior. O acesso a dados emprega o padrão Repository (FOWLER, 2006), por meio de uma interface genérica cuja implementação utiliza o Entity Framework Core, e as dependências são fornecidas por injeção, o que permitiu testar os serviços com repositórios simulados.

Quadro 6 – Camadas da solução e suas responsabilidades

Camada	Local no projeto	Responsabilidade
Apresentação	Frontend e Mobile	Interfaces do usuário, clientes das APIs
API	PataFacil/Controllers	Rotas REST, autenticação e autorização
Negócio (BLL)	PataFacil/BLL	Regras de agendamento, venda, prontuário e relatórios
Dados (DAL)	PataFacil/DAL	Repositório genérico, contexto e script do banco
Modelos	PataFacil/Models	Entidades e enumerações do domínio
Inteligência artificial	MachineLearning	Microsserviço de previsões em Python

Fonte: elaborado pelo autor (2026).

7.3 Modularização

O sistema é modularizado por domínio de negócio: agendamento, clínica, estoque e vendas, relatórios e autenticação possuem controllers e serviços próprios, de modo que uma alteração em um módulo não exige mexer nos demais. O repositório genérico elimina a repetição do código de acesso a dados entre as treze entidades persistidas.

A inteligência artificial constitui um módulo independente, implantado como microsserviço em Python com FastAPI e scikit-learn (PEDREGOSA et al., 2011) e acessado por HTTP. Ele oferece três modelos: classificação do risco de

cancelamento com floresta aleatória, previsão da demanda diária com regressão linear e segmentação de clientes com K-Means. Como a organização ainda não possui histórico digital, os modelos foram treinados com dados sintéticos que reproduzem o comportamento esperado do negócio; nesse conjunto, o classificador obteve acurácia de 0,74 e a regressão, coeficiente de determinação de 0,83. Esses valores servem para validar a integração e deverão ser reavaliados com dados reais.

8 PROJETO DO BANCO DE DADOS

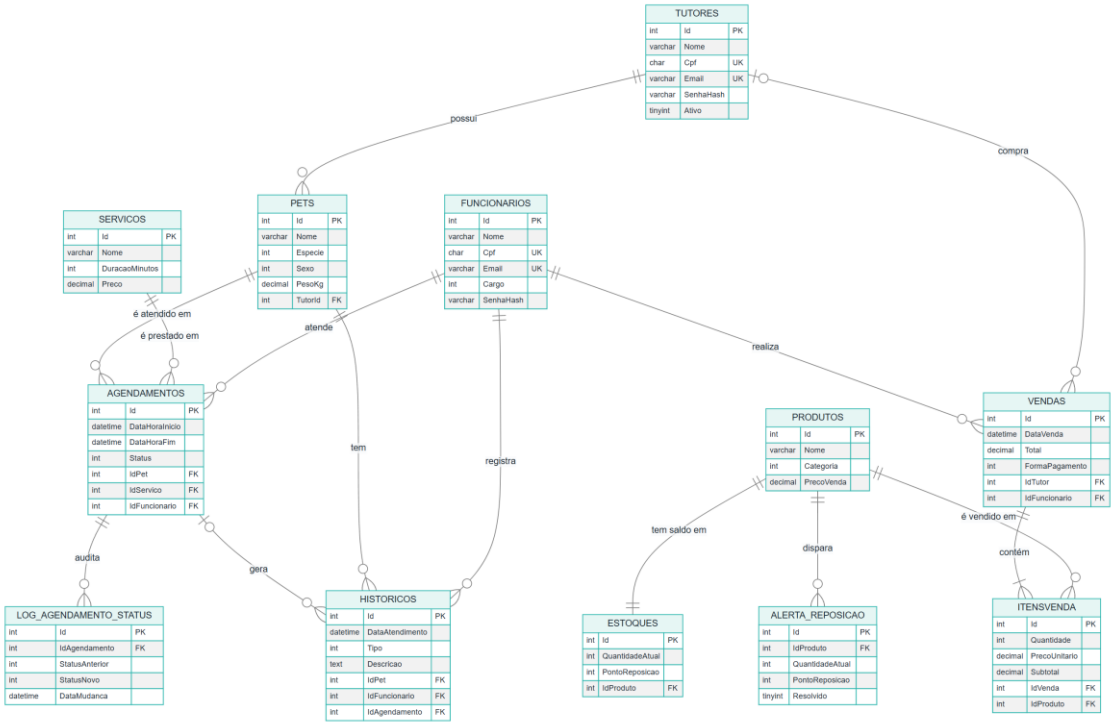
O banco de dados foi projetado no modelo relacional e implementado no MySQL 8 (ORACLE, 2026). O projeto seguiu as etapas clássicas de modelagem conceitual, lógica e física (ELMASRI; NAVATHE, 2018).

8.1 Modelo entidade-relacionamento

A Figura 13 apresenta o modelo entidade-relacionamento (MER) do núcleo do negócio, com dez entidades e as duas tabelas de apoio. Um tutor possui vários animais; cada agendamento associa um animal, um serviço e um profissional; o histórico clínico pertence a um animal e pode ter sido originado por um agendamento; cada produto tem um registro de estoque; e uma venda é composta por itens, cada qual referente a um produto.

Nas últimas Sprints o modelo ganhou três tabelas, descritas no Quadro 7: emails, que guarda as mensagens enviadas pelo sistema para a caixa de e-mail de cada usuário; entradas_estoque, em que cada recebimento de mercadoria forma um lote com nota fiscal, fornecedor, validade, custo e preço aplicado, ligado ao produto e ao funcionário que recebeu; e fornecedores, com nome único, CNPJ, telefone e e-mail. A entrada guarda o nome do fornecedor, o que preserva o histórico mesmo que o cadastro seja alterado depois.

Figura 13 – Modelo entidade-relacionamento do banco de dados



Fonte: elaborado pelo autor (2026).

8.2 Modelo lógico

No modelo lógico, cada entidade foi convertida em uma relação com chave primária numérica e os relacionamentos, em chaves estrangeiras, como resume o Quadro 7. As relações estão na terceira forma normal: não há grupos repetidos nem atributos dependentes de outros que não a chave. A única redundância intencional é o preço unitário gravado no item de venda, que preserva o valor praticado mesmo que o preço do produto mude depois.

Quadro 7 – Modelo lógico: relações, atributos principais e chaves

Relação	Atributos principais (PK = chave primária; FK = chave estrangeira)
tutores	Id (PK), Nome, Cpf, Email, Telefone, endereço, SenhaHash, Ativo
pets	Id (PK), Nome, Especie, Raca, Sexo, DataNascimento, PesoKg, TutorId (FK)
funcionarios	Id (PK), Nome, Cpf, Email, Cargo, SenhaHash, DataAdmissao, Ativo
servicos	Id (PK), Nome, Descricao, DuracaoMinutos, Preco, Ativo
agendamentos	Id (PK), DataHoraInicio, DataHoraFim, Status, Observacoes, ObservacoesAtendimento, DataChegada, ValorPago, FormaPagamento, DataPagamento, IdPet (FK), IdServico (FK), IdFuncionario (FK)
historicos	Id (PK), DataAtendimento, Tipo, Descricao, Diagnostico, Prescricao, ProximaVacina, IdPet (FK), IdFuncionario (FK), IdAgendamento (FK, opcional)
produtos	Id (PK), Nome, Categoria, Marca, PrecoCusto, PrecoVenda, Ativo
estoques	Id (PK), QuantidadeAtual, PontoReposicao, IdProduto (FK, único)
vendas	Id (PK), DataVenda, Total, FormaPagamento, IdTutor (FK, opcional), IdFuncionario (FK)

Relação	Atributos principais (PK = chave primária; FK = chave estrangeira)
itensvenda	Id (PK), Quantidade, PrecoUnitario, Subtotal, IdVenda (FK), IdProduto (FK)
log_agendamento_status	Id (PK), IdAgendamento (FK), StatusAnterior, StatusNovo, DataMudanca
alerta_reposicao	Id (PK), IdProduto (FK), QuantidadeAtual, PontoReposicao, Resolvido
emails	Id (PK), Destinatario, Assunto, Corpo, DataEnvio, Lida
entradas_estoque	Id (PK), IdProduto (FK), Quantidade, NumeroNota, Fornecedor, Validade, CustoUnitario, PrecoVendaAplicado, DataEntrada, IdFuncionario (FK)
fornecedores	Id (PK), Nome (único), Cnpj, Telefone, Email, DataCadastro, Ativo

Fonte: elaborado pelo autor (2026).

8.3 Modelo físico

No modelo físico foram definidos os tipos de dados, as restrições e os índices. Os textos usam VARCHAR com tamanho adequado ao conteúdo, os valores monetários usam DECIMAL(10,2) e as enumerações do código são gravadas como inteiros. Além das chaves primárias e estrangeiras, o esquema declara unicidade para e-mail e CPF e restrições de verificação que impedem, por exemplo, status fora do domínio, término anterior ao início, saldo de estoque negativo e entrada de mercadoria com preço de venda abaixo do custo. Dez índices otimizam as consultas mais frequentes; o índice composto por profissional e horário de início atende à verificação de conflito de agenda, o que foi confirmado pelo plano de execução da consulta.

8.4 Procedures

As cinco stored procedures de negócio do Quadro 8 encapsulam operações que envolvem várias tabelas e permitem que outras ferramentas, como planilhas e sistemas de relatórios, utilizem o banco respeitando as mesmas regras da aplicação. Duas procedures auxiliares, sp_garantir_coluna e sp_recriar_check, atualizam bancos criados por versões anteriores, acrescentando colunas e restrições sem apagar dados.

Quadro 8 – Stored procedures implementadas

Procedure	Função	Regras
sp_agendar_servico	Agenda um serviço validando data futura, expediente e conflito de horário; devolve o identificador e uma mensagem	RN01, RN03, RN10
sp_horarios_disponiveis	Lista os horários livres de um profissional em uma data, em intervalos de 15 minutos	RN01, RN03
sp_registrar_venda	Registra a venda e seus itens, recebidos em JSON, em uma única transação; desfaz tudo se faltar estoque	RN15
sp_receita_mensal	Totaliza vendas de produtos, ticket médio e serviços concluídos do mês	Gestão
sp_estoque_critico	Lista os produtos no ponto de reposição ou abaixo dele	RN15

Fonte: elaborado pelo autor (2026).

8.5 Triggers

As triggers do Quadro 9 garantem, no próprio banco, regras que não podem depender da aplicação que grava os dados. A baixa de estoque é feita exclusivamente pela trigger de itens de venda: a camada de negócio valida o saldo e calcula os alertas, mas não repete a baixa, o que evita o desconto em duplicidade.

Quadro 9 – Triggers implementadas

Trigger	Evento	Ação
trg_agendamento_calcula_fim	Antes de inserir em agendamentos	Calcula o término pela duração do serviço (RN10)
trg_item_venda_baixa_estoque	Após inserir em itensvenda	Baixa o estoque e gera o alerta de reposição (RN15)
trg_item_venda_atualiza_total	Após inserir em itensvenda	Recalcula o total da venda
trg_agendamento_log_status	Após atualizar agendamentos	Registra a mudança de status para auditoria (RN04)
trg_estoque_resolve_alerta	Após atualizar estoques	Encerra os alertas quando a reposição supera o ponto mínimo

Fonte: elaborado pelo autor (2026).

8.6 Script completo e validação

O script completo do banco, reproduzido no Apêndice A, cria as quinze tabelas, as cinco triggers, as sete procedures, as três views e os índices. Ele é aplicado pela própria API ao iniciar, de modo que a aplicação web, o aplicativo e as APIs utilizam exatamente a estrutura documentada, e pode ser executado repetidas vezes sem erro. O Quadro 10 resume os testes realizados diretamente no MySQL 8.0.

Quadro 10 – Resultados dos testes executados no banco de dados

Teste	Resultado obtido
Agendamento válido pela procedure	Criado, com mensagem de sucesso
Segundo agendamento sobreposto (RN01)	Recusado: conflito de horário com outro atendimento
Agendamento em domingo ou após as 19h (RN03)	Recusado, com a mensagem da regra violada
Inserção com término incorreto (RN10)	Trigger corrigiu o término para início mais 30 minutos
Mudança de status (RN04)	Registro da transição de 0 para 1 gravado na auditoria
Venda de 12 unidades com saldo 20 e ponto 10 (RN15)	Saldo 8, total de R$ 180,00 calculado e um alerta aberto
Venda acima do saldo	Recusada, com o saldo inalterado
Atualização que deixaria o saldo negativo	Bloqueada pela restrição de verificação
Reposição acima do ponto mínimo	Alerta encerrado automaticamente

Fonte: elaborado pelo autor (2026).

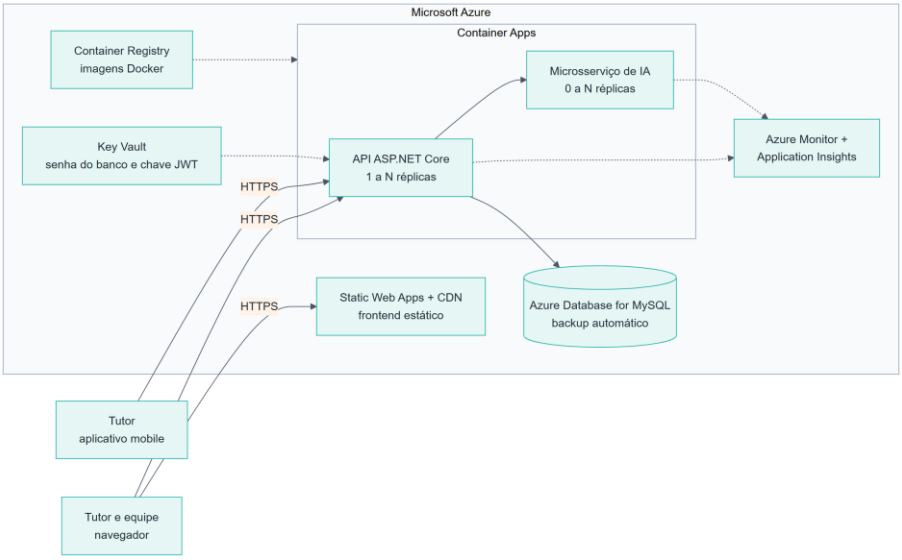
9 INFRAESTRUTURA EM NUVEM E DEVOPS

Este capítulo apresenta o plano de execução da infraestrutura para implantar a solução. A computação em nuvem oferece recursos sob demanda, com elasticidade e pagamento pelo uso (MELL; GRANCE, 2011), características adequadas a um negócio de pequeno porte, que não dispõe de equipe para manter servidores. Os artefatos de containers e de automação já fazem parte do repositório do projeto; a contratação do provedor é a etapa seguinte à entrega.

9.1 Arquitetura em nuvem

A arquitetura de referência, na Figura 14, utiliza serviços gerenciados. O frontend é distribuído como conteúdo estático; a API e o microsserviço de inteligência artificial executam em containers; e o banco de dados é um serviço gerenciado, acessível somente pela rede interna. O aplicativo e o navegador comunicam-se com a API por HTTPS.

Figura 14 – Arquitetura de referência em nuvem



Fonte: elaborado pelo autor (2026).

9.2 Serviços utilizados

O Quadro 11 indica os serviços previstos, tomando a Microsoft Azure como referência pela integração com a plataforma .NET (MICROSOFT, 2026b). Como a solução é empacotada em containers, pode ser implantada de forma equivalente em outros provedores.

Quadro 11 – Serviços de nuvem previstos

Componente	Serviço	Justificativa
Frontend	Azure Static Web Apps	Hospedagem de arquivos estáticos com CDN e HTTPS
API e IA	Azure Container Apps	Execução de containers com escala automática
Banco de dados	Azure Database for MySQL	Serviço gerenciado com backup e atualização automáticos
Imagens	Container Registry	Registro privado das imagens Docker
Segredos	Azure Key Vault	Guarda da senha do banco e da chave JWT
Observabilidade	Azure Monitor e Application Insights	Métricas, logs, rastreamento e alertas

Fonte: elaborado pelo autor (2026).

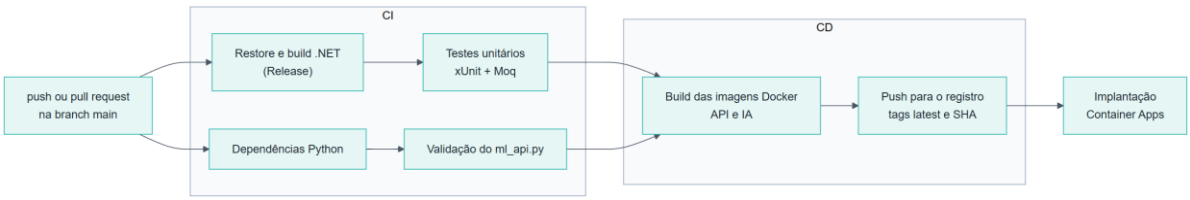
9.3 Containers

Cada componente possui sua imagem Docker (DOCKER, 2026). A imagem da API é construída em dois estágios: o primeiro compila e publica o projeto e o segundo contém apenas o ambiente de execução, o que reduz o tamanho e a superfície de ataque. Um arquivo docker-compose sobe o banco de dados, o microsserviço de inteligência artificial, a API e o servidor web com um único comando, com verificação de saúde do banco e do microsserviço e com a senha e a chave JWT fornecidas por variáveis de ambiente. Assim, o ambiente de desenvolvimento reproduz o de produção.

9.4 Pipeline de CI/CD

A integração e a entrega contínuas, práticas centrais da cultura DevOps (KIM et al., 2018), foram automatizadas com o GitHub Actions, como ilustra a Figura 15. A cada envio de código, o pipeline compila a API, executa os 28 testes unitários e valida o microsserviço em Python. Somente se essas etapas forem aprovadas, e apenas na ramificação principal, as imagens Docker são construídas e publicadas no registro, identificadas pelo código da versão, ficando prontas para a implantação.

Figura 15 – Pipeline de integração e entrega contínuas



Fonte: elaborado pelo autor (2026).

9.5 Monitoramento

O monitoramento combina três recursos. A API e o microsserviço expõem endpoints de verificação de saúde, utilizados pela plataforma para reiniciar instâncias com falha; os registros de execução são centralizados para consulta; e a ferramenta de observabilidade coleta tempo de resposta, taxa de erros e uso de recursos. Propõem-se alertas para indisponibilidade do banco ou da inteligência artificial e para tempo de resposta acima de dois segundos.

9.6 Escalabilidade

A API não mantém estado entre requisições, pois a identidade do usuário está no token; por isso, pode ser replicada horizontalmente atrás de um balanceador de carga. O microsserviço de inteligência artificial pode reduzir-se a zero instâncias nos períodos sem uso, o que diminui o custo. O banco gerenciado admite ampliação de capacidade e réplicas de leitura para os relatórios, e o frontend é servido por rede de distribuição de conteúdo.

9.7 Segurança

Na infraestrutura, a segurança complementa as medidas da aplicação: todo o tráfego externo utiliza HTTPS; o banco de dados não é exposto à internet; os segredos ficam em cofre e são injetados como variáveis de ambiente, nunca gravados no código; as imagens contêm apenas o necessário à execução; e o banco gerenciado realiza cópias de segurança automáticas, com retenção configurável, para atender à continuidade do negócio.

10 GERENCIAMENTO ÁGIL DO PROJETO

O desenvolvimento foi planejado e acompanhado com o Scrum, framework em que o produto é construído em ciclos curtos, com inspeção e adaptação frequentes (SCHWABER; SUTHERLAND, 2020). A abordagem é indicada quando os requisitos evoluem ao longo do projeto e se deseja entregar valor desde cedo (PRESSMAN; MAXIM, 2016).

10.1 Papéis e eventos

Foram definidos os três papéis do Scrum: o Product Owner, que representa os interesses da organização e ordena o backlog; o Scrum Master, que zela pelo método e remove impedimentos; e os desenvolvedores, responsáveis por backend, frontend, mobile, banco de dados e inteligência artificial. Cada Sprint, de duas semanas, compreendeu o planejamento, as reuniões diárias de quinze minutos, a revisão do incremento e a retrospectiva.

10.2 Product Backlog

O Product Backlog reúne as funcionalidades na forma de histórias de usuário, ordenadas pelo valor para o negócio. O Quadro 12 apresenta as 24 histórias, com a prioridade e a Sprint em que cada uma foi planejada. As histórias PB18 a PB24 surgiram nas revisões das Sprints, a partir do uso do sistema pela equipe do balcão, e foram atendidas na Sprint 7.

Quadro 12 – Product Backlog

ID	História de usuário	Prior.	Sprint
PB01	Como usuário, quero me autenticar para acessar meu painel com segurança	1	1
PB02	Como tutor, quero me cadastrar para criar minha conta	1	1
PB03	Como tutor, quero cadastrar meus pets para agendar serviços para eles	1	2
PB04	Como tutor, quero agendar serviços em passos simples para reservar horário sem telefonar	2	2
PB05	Como estabelecimento, quero impedir conflito de horário do profissional (RN01)	2	2
PB06	Como estabelecimento, quero limitar a agenda ao expediente e calcular a duração (RN03, RN10)	2	2
PB07	Como tutor, quero cancelar on-line com antecedência (RN02)	2	3
PB08	Como atendente, quero confirmar e concluir atendimentos em etapas travadas (RN04)	3	3
PB09	Como veterinário, quero registrar o prontuário para manter o histórico clínico	3	3
PB10	Como atendente, quero registrar vendas com carrinho de produtos	4	4

ID	História de usuário	Prior.	Sprint
PB11	Como estabelecimento, quero baixar o estoque a cada venda e ser alertado da reposição (RN15)	4	4
PB12	Como gerente, quero relatórios e indicadores para decidir com base em dados	5	5
PB13	Como gerente, quero cadastrar a equipe e controlar os acessos	5	5
PB14	Como gerente, quero prever demanda e risco de cancelamento	6	6
PB15	Como tutor, quero usar um aplicativo no celular para agendar de qualquer lugar	6	6
PB16	Como pessoa surda, quero conteúdo em Libras para usar o sistema com autonomia	7	6
PB17	Como equipe técnica, quero containers e pipeline para implantar com segurança	7	6
PB18	Como atendente, quero registrar a chegada do cliente e receber o pagamento com troco	3	7
PB19	Como usuário, quero uma caixa de e-mail com as mensagens enviadas pelo sistema	5	7
PB20	Como atendente, quero registrar a entrada de mercadoria com nota, validade e margem de lucro	4	7
PB21	Como atendente, quero escolher um fornecedor cadastrado ou cadastrar um novo na hora	4	7
PB22	Como atendente, quero emitir a ordem de serviço em A4, PDF ou impressora térmica	3	7
PB23	Como gerente, quero gráficos do mês e sugestões de promoção na visão geral	5	7
PB24	Como usuário, quero uma interface padronizada e de leitura confortável	6	7

Fonte: elaborado pelo autor (2026).

10.3 Sprint Backlog e cronograma das Sprints

O trabalho foi distribuído em sete Sprints, totalizando catorze semanas. No planejamento de cada Sprint, as histórias selecionadas foram decompostas em tarefas técnicas, que formam o Sprint Backlog. O Quadro 13 traz o cronograma, com o objetivo e o Sprint Backlog resumido de cada ciclo.

Quadro 13 – Cronograma das Sprints e Sprint Backlog resumido

Sprint	Período	Objetivo	Sprint Backlog (tarefas principais)
1	Semanas 1 e 2	Fundação e autenticação	Estrutura em camadas; repositório genérico; modelo de dados; login com JWT e BCrypt; cadastro de tutor
2	Semanas 3 e 4	Pets e agenda	Cadastro de pets; cálculo de horários livres; regras RN01, RN03 e RN10; assistente de agendamento web
3	Semanas 5 e 6	Fluxo clínico	Máquina de status (RN04); cancelamento (RN02); prontuário; painéis do atendente e do veterinário
4	Semanas 7 e 8	Estoque e vendas	Produtos e estoque; venda com carrinho; triggers de baixa e de alerta (RN15)
5	Semanas 9 e 10	Gestão	Indicadores; relatório de faturamento; gestão da equipe; procedures e views
6	Semanas 11 e 12	IA, mobile e DevOps	Microsserviço de IA; aplicativo; VLibras; containers; pipeline; testes automatizados
7	Semanas 13 e 14	Operação do balcão	Chegada e pagamento; caixa de e-mail; entrada de mercadoria; fornecedores; ordem de serviço; gráficos do mês; padronização visual; catálogo de demonstração

Fonte: elaborado pelo autor (2026).

10.4 Quadro Kanban

O andamento das tarefas foi acompanhado em um quadro Kanban, que torna o fluxo de trabalho visível e limita o trabalho em progresso (ANDERSON, 2011). Adotou-se o limite de três itens na coluna em andamento. O Quadro 14 reproduz a situação do quadro ao final da primeira semana da Sprint 4.

Quadro 14 – Quadro Kanban na primeira semana da Sprint 4

A fazer	Em andamento (limite 3)	Em revisão	Concluído
PB12 Relatórios e indicadores PB13 Gestão da equipe PB14 Previsões por IA PB15 Aplicativo mobile PB16 Libras PB17 Containers e pipeline	PB11 Trigger de baixa de estoque PB11 Alerta de reposição	PB10 Venda com carrinho	PB01 a PB09 (Sprints 1 a 3) PB10 Cadastro de produtos

Fonte: elaborado pelo autor (2026).

10.5 Definição das entregas

Cada Sprint resultou em um incremento utilizável, descrito no Quadro 15. Uma história só foi considerada pronta quando atendeu à definição de pronto acordada: código compilando e integrado; regras de negócio verificadas por teste; funcionamento de ponta a ponta na interface correspondente; e aceite do Product Owner na revisão da Sprint.

Quadro 15 – Entregas por Sprint

Sprint	Incremento entregue
1	API com autenticação por token e cadastro de tutor
2	Agendamento web sem conflitos de horário, dentro do expediente
3	Ciclo completo de atendimento, do agendamento ao prontuário
4	Vendas integradas ao estoque, com alerta de reposição
5	Painel gerencial com indicadores e relatório de faturamento
6	Solução integrada: web, aplicativo, inteligência artificial, containers e pipeline
7	Balcão completo, do recebimento do cliente à ordem de serviço, e compras com fornecedores cadastrados

Fonte: elaborado pelo autor (2026).

11 CONCLUSÃO

Este trabalho propôs desenvolver e validar uma solução tecnológica integrada para apoiar a transformação digital de um pet shop com clínica veterinária, em continuidade ao PIM III. O objetivo foi alcançado: o PataFácil reúne, em uma única plataforma, a agenda de serviços, o prontuário clínico, o estoque, as vendas e os indicadores gerenciais, com acesso pela web e por aplicativo móvel sobre as mesmas APIs.

Os problemas identificados na caracterização da organização receberam resposta direta. O conflito de horários é impedido pela validação de sobreposição; o tutor ganhou autonomia para agendar e cancelar; o histórico clínico passou a ser digital; a venda baixa o estoque e alerta a reposição; o recebimento de mercadorias registra nota, validade e fornecedor; cada atendimento gera sua ordem de serviço para impressão, PDF ou cupom térmico; e o gerente dispõe de indicadores e de previsões. A arquitetura em camadas e orientada a objetos concentrou as regras no servidor e no banco de dados, o que garantiu o mesmo comportamento nos dois canais, e os testes automatizados e de integração, todos aprovados, sustentam a confiança no resultado.

O projeto também evidenciou limites, que orientam os próximos passos. Os modelos de inteligência artificial foram treinados com dados sintéticos e precisam ser reavaliados quando houver histórico real; o envio de e-mails está simulado e deve ser ligado a um serviço transacional; e a infraestrutura em nuvem está planejada e automatizada, mas ainda não contratada. Recomenda-se, ainda, realizar testes de acessibilidade com usuários e incluir lembretes automáticos de vacinas e de consultas.

Conclui-se que a integração dos conhecimentos de empreendedorismo, desenvolvimento web e mobile, arquitetura de software, banco de dados, computação em nuvem, DevOps e gerenciamento ágil permitiu entregar uma solução viável, segura e inclusiva, capaz de aumentar a eficiência e a competitividade de um pequeno negócio, o que reafirma o papel da tecnologia como instrumento de transformação das organizações.

REFERÊNCIAS

ALMEIDA, Silvio Luiz de. **Racismo estrutural**. São Paulo: Sueli Carneiro; Pólen, 2019.

ANDERSON, David J. **Kanban**: mudança evolucionária de sucesso para seu negócio de tecnologia. [S. l.]: Blue Hole Press, 2011.

BRASIL. **Lei nº 10.639, de 9 de janeiro de 2003**. Altera a Lei nº 9.394, de 20 de dezembro de 1996, para incluir no currículo oficial da rede de ensino a obrigatoriedade da temática “História e Cultura Afro-Brasileira”. Brasília, DF: Presidência da República, 2003. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/2003/l10.639.htm. Acesso em: 10 out. 2026.

BRASIL. **Lei nº 12.288, de 20 de julho de 2010**. Institui o Estatuto da Igualdade Racial. Brasília, DF: Presidência da República, 2010. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2010/lei/l12288.htm. Acesso em: 10 out. 2026.

BRASIL. **Lei nº 13.146, de 6 de julho de 2015**. Institui a Lei Brasileira de Inclusão da Pessoa com Deficiência (Estatuto da Pessoa com Deficiência). Brasília, DF: Presidência da República, 2015. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2015/lei/l13146.htm. Acesso em: 10 out. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 10 out. 2026.

BRASIL. Ministério da Gestão e da Inovação em Serviços Públicos. **VLibras**. Brasília, DF, 2026. Disponível em: https://www.gov.br/governodigital/pt-br/acessibilidade-e-usuario/vlibras. Acesso em: 10 out. 2026.

DOCKER. **Docker documentation**. [S. l.], 2026. Disponível em: https://docs.docker.com. Acesso em: 10 out. 2026.

DORNELAS, José. **Empreendedorismo**: transformando ideias em negócios. 7. ed. São Paulo: Empreende, 2018.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson Education do Brasil, 2018.

EXPO. **Expo documentation**. [S. l.], 2026. Disponível em: https://docs.expo.dev. Acesso em: 10 out. 2026.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Informação e Ciência da Computação) – University of California, Irvine, 2000. Disponível em: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm. Acesso em: 10 out. 2026.

FOWLER, Martin. **Padrões de arquitetura de aplicações corporativas**. Porto Alegre: Bookman, 2006.

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 6. ed. São Paulo: Atlas, 2017.

JONES, Michael; BRADLEY, John; SAKIMURA, Nat. **JSON Web Token (JWT)**: RFC 7519. [S. l.]: Internet Engineering Task Force, 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7519. Acesso em: 10 out. 2026.

KIM, Gene et al. **Manual de DevOps**: como obter agilidade, confiabilidade e segurança em organizações tecnológicas. Rio de Janeiro: Alta Books, 2018.

MARCONI, Marina de Andrade; LAKATOS, Eva Maria. **Fundamentos de metodologia científica**. 8. ed. São Paulo: Atlas, 2017.

MELL, Peter; GRANCE, Timothy. **The NIST definition of cloud computing**. Gaithersburg: National Institute of Standards and Technology, 2011. (Special Publication 800-145). Disponível em: https://doi.org/10.6028/NIST.SP.800-145. Acesso em: 10 out. 2026.

META PLATFORMS. **React Native documentation**. [S. l.], 2026. Disponível em: https://reactnative.dev/docs/getting-started. Acesso em: 10 out. 2026.

MICROSOFT. **Documentação do ASP.NET Core**. [S. l.], 2026a. Disponível em: https://learn.microsoft.com/pt-br/aspnet/core. Acesso em: 10 out. 2026.

MICROSOFT. **Documentação dos Aplicativos de Contêiner do Azure**. [S. l.], 2026b. Disponível em: https://learn.microsoft.com/pt-br/azure/container-apps. Acesso em: 10 out. 2026.

MUNANGA, Kabengele; GOMES, Nilma Lino. **O negro no Brasil de hoje**. São Paulo: Global, 2006.

ORACLE. **MySQL 8.0 reference manual**. [S. l.], 2026. Disponível em: https://dev.mysql.com/doc/refman/8.0/en. Acesso em: 10 out. 2026.

OSTERWALDER, Alexander; PIGNEUR, Yves. **Business model generation**: inovação em modelos de negócios. Rio de Janeiro: Alta Books, 2011.

PEDREGOSA, Fabian et al. Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, [s. l.], v. 12, p. 2825-2830, 2011.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SCHWABER, Ken; SUTHERLAND, Jeff. **O guia do Scrum**: o guia definitivo para o Scrum: as regras do jogo. [S. l.], 2020. Disponível em: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-PortugueseBR-3.0.pdf. Acesso em: 10 out. 2026.

SEVERINO, Antônio Joaquim. **Metodologia do trabalho científico**. 24. ed. São Paulo: Cortez, 2016.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.1**. [S. l.]: World Wide Web Consortium, 2018. Disponível em: https://www.w3.org/TR/WCAG21. Acesso em: 10 out. 2026.

APÊNDICE A – SCRIPT COMPLETO DO BANCO DE DADOS

Script executado pela aplicação na inicialização (arquivo PataFacil/DAL/ScriptBancoDeDados.sql). Contém as tabelas, as triggers, as procedures, as views e os índices utilizados pelas APIs, pela aplicação web e pelo aplicativo mobile.

```
-- ============================================================
-- PATAFÁCIL — SCRIPT COMPLETO DO BANCO DE DADOS (PIM IV)
-- Etapa 7: Projeto do Banco de Dados — MySQL 8
-- ============================================================
-- Este é o banco usado pelas APIs, pela aplicação Web e pelo Mobile.
-- A própria API aplica este script ao iniciar (DAL/InicializadorBanco.cs),
-- portanto ele pode ser executado várias vezes sem erro (idempotente).
--
-- Execução manual, se desejar:
--   mysql -u root -p < ScriptBancoDeDados.sql
--
-- Convenção: os enums do C# são gravados como INT (os valores estão
-- comentados em cada coluna).

CREATE DATABASE IF NOT EXISTS patafacil
  DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE patafacil;

-- ============================================================
-- 1. TABELAS (MODELO FÍSICO)
-- ============================================================

-- Tabela 1: Tutores (especialização de Pessoa)
CREATE TABLE IF NOT EXISTS tutores (
    Id INT NOT NULL AUTO_INCREMENT,
    Nome VARCHAR(120) NOT NULL,
    Cpf CHAR(11) NOT NULL,
    Email VARCHAR(160) NOT NULL,
    Telefone VARCHAR(20) NOT NULL DEFAULT '',
    Logradouro VARCHAR(160) NOT NULL DEFAULT '',
    Numero VARCHAR(10) NOT NULL DEFAULT '',
    Bairro VARCHAR(80) NOT NULL DEFAULT '',
    Cidade VARCHAR(80) NOT NULL DEFAULT '',
    Uf CHAR(2) NOT NULL DEFAULT '',
    Cep VARCHAR(9) NOT NULL DEFAULT '',
    DataCadastro DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    SenhaHash VARCHAR(100) NOT NULL,          -- hash BCrypt (nunca a senha em texto puro)
    Ativo TINYINT(1) NOT NULL DEFAULT 1,
    CONSTRAINT PK_tutores PRIMARY KEY (Id),
    CONSTRAINT UQ_tutores_email UNIQUE (Email),
    CONSTRAINT UQ_tutores_cpf UNIQUE (Cpf)
) ENGINE=InnoDB;

-- Tabela 2: Pets
CREATE TABLE IF NOT EXISTS pets (
    Id INT NOT NULL AUTO_INCREMENT,
    Nome VARCHAR(80) NOT NULL,
    Especie INT NOT NULL,                     -- 0=Cachorro, 1=Gato, 2=Ave, 3=Roedor, 4=Outro
    Raca VARCHAR(80) NOT NULL DEFAULT '',
    Sexo INT NOT NULL,                        -- 0=Macho, 1=Femea
    DataNascimento DATETIME(6) NOT NULL,
    PesoKg DECIMAL(6,2) NOT NULL DEFAULT 0,
    FotoUrl VARCHAR(255) NOT NULL DEFAULT '',
    Observacoes TEXT NOT NULL,
    Ativo TINYINT(1) NOT NULL DEFAULT 1,
    TutorId INT NOT NULL,
    CONSTRAINT PK_pets PRIMARY KEY (Id),
    CONSTRAINT FK_pets_tutores FOREIGN KEY (TutorId) REFERENCES tutores(Id),
    CONSTRAINT CK_pets_especie CHECK (Especie BETWEEN 0 AND 4),
    CONSTRAINT CK_pets_sexo CHECK (Sexo IN (0, 1)),
    INDEX idx_pets_tutor (TutorId)
) ENGINE=InnoDB;

-- Tabela 3: Funcionarios (especialização de Pessoa)
CREATE TABLE IF NOT EXISTS funcionarios (
```

```
    Id INT NOT NULL AUTO_INCREMENT,
    Nome VARCHAR(120) NOT NULL,
    Cpf CHAR(11) NOT NULL,
    Email VARCHAR(160) NOT NULL,
    Cargo INT NOT NULL,                         -- 0=Veterinario, 1=Atendente, 2=Gerente,
3=Tosador (Banho e Tosa)
    SenhaHash VARCHAR(100) NOT NULL,            -- hash BCrypt
    DataAdmissao DATETIME(6) NOT NULL,
    Ativo TINYINT(1) NOT NULL DEFAULT 1,
    CONSTRAINT PK_funcionarios PRIMARY KEY (Id),
    CONSTRAINT UQ_funcionarios_email UNIQUE (Email),
    CONSTRAINT UQ_funcionarios_cpf UNIQUE (Cpf),
    CONSTRAINT CK_funcionarios_cargo CHECK (Cargo BETWEEN 0 AND 3)
) ENGINE=InnoDB;


-- Tabela 4: Servicos
CREATE TABLE IF NOT EXISTS servicos (
    Id INT NOT NULL AUTO_INCREMENT,
    Nome VARCHAR(80) NOT NULL,
    Descricao VARCHAR(255) NOT NULL DEFAULT '',
    DuracaoMinutos INT NOT NULL,
    Preco DECIMAL(10,2) NOT NULL,
    Ativo TINYINT(1) NOT NULL DEFAULT 1,
    CONSTRAINT PK_servicos PRIMARY KEY (Id),
    CONSTRAINT CK_servicos_duracao CHECK (DuracaoMinutos > 0),
    CONSTRAINT CK_servicos_preco CHECK (Preco >= 0)
) ENGINE=InnoDB;


-- Tabela 5: Agendamentos
CREATE TABLE IF NOT EXISTS agendamentos (
    Id INT NOT NULL AUTO_INCREMENT,
    DataHoraInicio DATETIME(6) NOT NULL,
    DataHoraFim DATETIME(6) NOT NULL,
    Observacoes TEXT NULL,
    DataCriacao DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    IdPet INT NOT NULL,
    IdServico INT NOT NULL,
    IdFuncionario INT NOT NULL,
    Status INT NOT NULL DEFAULT 0,              -- 0=PENDENTE, 1=CONFIRMADO, 2=CONCLUIDO,
3=CANCELADO
    ObservacoesAtendimento TEXT NULL,           -- anotações internas da equipe sobre o pet
    DataChegada DATETIME(6) NULL,               -- preenchida quando o cliente chega ao balcão
("em processo")
    ValorPago DECIMAL(10,2) NULL,               -- baixa no balcão
    FormaPagamento INT NULL,                    -- 0=Dinheiro, 1=CartaoCredito, 2=CartaoDebito,
3=Pix
    DataPagamento DATETIME(6) NULL,
    CONSTRAINT PK_agendamentos PRIMARY KEY (Id),
    CONSTRAINT FK_agendamentos_pets FOREIGN KEY (IdPet) REFERENCES pets(Id),
    CONSTRAINT FK_agendamentos_servicos FOREIGN KEY (IdServico) REFERENCES servicos(Id),
    CONSTRAINT FK_agendamentos_funcionarios FOREIGN KEY (IdFuncionario) REFERENCES
funcionarios(Id),
    CONSTRAINT CK_agendamentos_status CHECK (Status BETWEEN 0 AND 3),
    CONSTRAINT CK_agendamentos_periodo CHECK (DataHoraFim > DataHoraInicio),
    -- Índices de otimização: checagem de conflito (RN01), agenda do dia/semana e filtros por
status
    INDEX idx_agendamentos_funcionario_horario (IdFuncionario, DataHoraInicio),
    INDEX idx_agendamentos_data (DataHoraInicio),
    INDEX idx_agendamentos_status (Status)
) ENGINE=InnoDB;


-- Tabela 6: Historicos (Histórico Clínico)
CREATE TABLE IF NOT EXISTS historicos (
    Id INT NOT NULL AUTO_INCREMENT,
    DataAtendimento DATETIME(6) NOT NULL,
    Tipo INT NOT NULL,                          -- 0=Consulta, 1=Vacina, 2=Cirurgia, 3=Retorno,
4=Exame
    Descricao TEXT NOT NULL,
    Diagnostico TEXT NULL,
    Prescricao TEXT NULL,
    PesoRegistro DECIMAL(6,2) NOT NULL DEFAULT 0,
    ProximaVacina DATETIME(6) NULL,
    IdPet INT NOT NULL,
    IdFuncionario INT NOT NULL,
    IdAgendamento INT NULL,                     -- opcional: nem todo registro nasce de um
agendamento
    CONSTRAINT PK_historicos PRIMARY KEY (Id),
    CONSTRAINT FK_historicos_pets FOREIGN KEY (IdPet) REFERENCES pets(Id),
    CONSTRAINT FK_historicos_funcionarios FOREIGN KEY (IdFuncionario) REFERENCES
funcionarios(Id),
```

```
    CONSTRAINT FK_historicos_agendamentos FOREIGN KEY (IdAgendamento) REFERENCES
agendamentos(Id),
    CONSTRAINT CK_historicos_tipo CHECK (Tipo BETWEEN 0 AND 4),
    INDEX idx_historicos_pet (IdPet, DataAtendimento)
) ENGINE=InnoDB;


-- Tabela 7: Produtos
CREATE TABLE IF NOT EXISTS produtos (
    Id INT NOT NULL AUTO_INCREMENT,
    Nome VARCHAR(120) NOT NULL,
    Descricao VARCHAR(255) NOT NULL DEFAULT '',
    Categoria INT NOT NULL DEFAULT 0,         -- 0=Alimento, 1=Medicamento, 2=Acessorio,
3=Higiene, 4=Outro
    Marca VARCHAR(80) NOT NULL DEFAULT '',
    PrecoCusto DECIMAL(10,2) NOT NULL DEFAULT 0,
    PrecoVenda DECIMAL(10,2) NOT NULL,
    UnidadeMedida VARCHAR(10) NOT NULL DEFAULT '',
    CodigoBarras VARCHAR(20) NOT NULL DEFAULT '',
    Ativo TINYINT(1) NOT NULL DEFAULT 1,
    CONSTRAINT PK_produtos PRIMARY KEY (Id),
    CONSTRAINT CK_produtos_categoria CHECK (Categoria BETWEEN 0 AND 4),
    CONSTRAINT CK_produtos_preco CHECK (PrecoVenda >= 0 AND PrecoCusto >= 0)
) ENGINE=InnoDB;


-- Tabela 8: Estoques (um registro de saldo por produto)
CREATE TABLE IF NOT EXISTS estoques (
    Id INT NOT NULL AUTO_INCREMENT,
    QuantidadeAtual INT NOT NULL DEFAULT 0,
    PontoReposicao INT NOT NULL DEFAULT 0,
    QuantidadeReposicao INT NOT NULL DEFAULT 0,
    UltimaAtualizacao DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    IdProduto INT NOT NULL,
    CONSTRAINT PK_estoques PRIMARY KEY (Id),
    CONSTRAINT FK_estoques_produtos FOREIGN KEY (IdProduto) REFERENCES produtos(Id),
    CONSTRAINT UQ_estoques_produto UNIQUE (IdProduto),
    -- Última barreira contra venda sem saldo: o banco recusa estoque negativo
    CONSTRAINT CK_estoques_saldo CHECK (QuantidadeAtual >= 0)
) ENGINE=InnoDB;


-- Tabela 9: Vendas
CREATE TABLE IF NOT EXISTS vendas (
    Id INT NOT NULL AUTO_INCREMENT,
    DataVenda DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    Total DECIMAL(10,2) NOT NULL DEFAULT 0,
    FormaPagamento INT NOT NULL,              -- 0=Dinheiro, 1=CartaoCredito, 2=CartaoDebito,
3=Pix
    Observacoes TEXT NULL,
    IdTutor INT NULL,                         -- opcional: venda de balcão sem cliente
cadastrado
    IdFuncionario INT NOT NULL,
    CONSTRAINT PK_vendas PRIMARY KEY (Id),
    CONSTRAINT FK_vendas_tutores FOREIGN KEY (IdTutor) REFERENCES tutores(Id),
    CONSTRAINT FK_vendas_funcionarios FOREIGN KEY (IdFuncionario) REFERENCES funcionarios(Id),
    CONSTRAINT CK_vendas_pagamento CHECK (FormaPagamento BETWEEN 0 AND 3),
    INDEX idx_vendas_data (DataVenda)
) ENGINE=InnoDB;


-- Tabela 10: ItensVenda
CREATE TABLE IF NOT EXISTS itensvenda (
    Id INT NOT NULL AUTO_INCREMENT,
    Quantidade INT NOT NULL,
    PrecoUnitario DECIMAL(10,2) NOT NULL,
    Subtotal DECIMAL(10,2) NOT NULL,
    IdVenda INT NOT NULL,
    IdProduto INT NOT NULL,
    CONSTRAINT PK_itensvenda PRIMARY KEY (Id),
    CONSTRAINT FK_itensvenda_vendas FOREIGN KEY (IdVenda) REFERENCES vendas(Id),
    CONSTRAINT FK_itensvenda_produtos FOREIGN KEY (IdProduto) REFERENCES produtos(Id),
    CONSTRAINT CK_itensvenda_quantidade CHECK (Quantidade > 0),
    INDEX idx_itensvenda_produto (IdProduto)
) ENGINE=InnoDB;


-- ============================================================
-- 2. TABELAS DE APOIO (AUDITORIA E ALERTAS)
-- ============================================================


-- Log de auditoria do status do agendamento (apoia a RN04)
CREATE TABLE IF NOT EXISTS log_agendamento_status (
```

```
    Id INT NOT NULL AUTO_INCREMENT,
    IdAgendamento INT NOT NULL,
    StatusAnterior INT NULL,
    StatusNovo INT NOT NULL,
    DataMudanca DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    CONSTRAINT PK_log_agendamento_status PRIMARY KEY (Id),
    CONSTRAINT FK_log_status_agendamento FOREIGN KEY (IdAgendamento) REFERENCES
agendamentos(Id)
) ENGINE=InnoDB;


-- Alerta automático de reposição (apoia a RN15)
CREATE TABLE IF NOT EXISTS alerta_reposicao (
    Id INT NOT NULL AUTO_INCREMENT,
    IdProduto INT NOT NULL,
    QuantidadeAtual INT NOT NULL,
    PontoReposicao INT NOT NULL,
    DataAlerta DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    Resolvido TINYINT(1) NOT NULL DEFAULT 0,
    CONSTRAINT PK_alerta_reposicao PRIMARY KEY (Id),
    CONSTRAINT FK_alerta_produto FOREIGN KEY (IdProduto) REFERENCES produtos(Id)
) ENGINE=InnoDB;


-- Caixa de E-mail: e-mails automáticos enviados pelo sistema (RF09)
CREATE TABLE IF NOT EXISTS emails (
    Id INT NOT NULL AUTO_INCREMENT,
    Destinatario VARCHAR(160) NOT NULL,
    Assunto VARCHAR(200) NOT NULL,
    Corpo TEXT NOT NULL,
    DataEnvio DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    Lida TINYINT(1) NOT NULL DEFAULT 0,
    CONSTRAINT PK_emails PRIMARY KEY (Id),
    INDEX idx_emails_destinatario (Destinatario, DataEnvio)
) ENGINE=InnoDB;


-- Entradas de mercadoria (compra do fornecedor): nota, custo e validade de cada lote recebido
CREATE TABLE IF NOT EXISTS entradas_estoque (
    Id INT NOT NULL AUTO_INCREMENT,
    IdProduto INT NOT NULL,
    Quantidade INT NOT NULL,
    NumeroNota VARCHAR(30) NOT NULL,
    Fornecedor VARCHAR(120) NOT NULL DEFAULT '',
    Validade DATETIME(6) NULL,                -- opcional: nem todo produto vence
    CustoUnitario DECIMAL(10,2) NOT NULL,
    PrecoVendaAplicado DECIMAL(10,2) NOT NULL,
    DataEntrada DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    IdFuncionario INT NOT NULL,
    CONSTRAINT PK_entradas_estoque PRIMARY KEY (Id),
    CONSTRAINT FK_entradas_produto FOREIGN KEY (IdProduto) REFERENCES produtos(Id),
    CONSTRAINT FK_entradas_funcionario FOREIGN KEY (IdFuncionario) REFERENCES
funcionarios(Id),
    CONSTRAINT CK_entradas_quantidade CHECK (Quantidade > 0),
    CONSTRAINT CK_entradas_valores CHECK (CustoUnitario > 0 AND PrecoVendaAplicado >=
CustoUnitario),
    INDEX idx_entradas_produto (IdProduto, Validade),
    INDEX idx_entradas_data (DataEntrada)
) ENGINE=InnoDB;


-- Fornecedores cadastrados (a entrada de mercadoria guarda o nome do fornecedor)
CREATE TABLE IF NOT EXISTS fornecedores (
    Id INT NOT NULL AUTO_INCREMENT,
    Nome VARCHAR(120) NOT NULL,
    Cnpj VARCHAR(18) NOT NULL DEFAULT '',
    Telefone VARCHAR(20) NOT NULL DEFAULT '',
    Email VARCHAR(120) NOT NULL DEFAULT '',
    DataCadastro DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    Ativo TINYINT(1) NOT NULL DEFAULT 1,
    CONSTRAINT PK_fornecedores PRIMARY KEY (Id),
    CONSTRAINT UQ_fornecedores_nome UNIQUE (Nome)
) ENGINE=InnoDB;


-- Compatibilidade: bancos criados por versões anteriores tinham estas duas colunas como
-- NOT NULL. Em um banco novo os comandos abaixo não alteram nada.
ALTER TABLE vendas MODIFY IdTutor INT NULL;
ALTER TABLE historicos MODIFY IdAgendamento INT NULL;


-- Atualização de bancos já existentes: o MySQL não tem "ADD COLUMN IF NOT EXISTS",
-- então estas rotinas consultam o information_schema antes de alterar a tabela.
DROP PROCEDURE IF EXISTS sp_garantir_coluna;
```

```
DROP PROCEDURE IF EXISTS sp_recriar_check;

DELIMITER $$

CREATE PROCEDURE sp_garantir_coluna(IN p_tabela VARCHAR(64), IN p_coluna VARCHAR(64), IN
p_definicao VARCHAR(255))
BEGIN
    IF NOT EXISTS (SELECT 1 FROM information_schema.COLUMNS
                   WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = p_tabela AND COLUMN_NAME =
p_coluna) THEN
        SET @ddl = CONCAT('ALTER TABLE ', p_tabela, ' ADD COLUMN ', p_coluna, ' ',
p_definicao);
        PREPARE comando FROM @ddl;
        EXECUTE comando;
        DEALLOCATE PREPARE comando;
    END IF;
END$$

CREATE PROCEDURE sp_recriar_check(IN p_tabela VARCHAR(64), IN p_nome VARCHAR(64), IN
p_expressao VARCHAR(255))
BEGIN
    IF EXISTS (SELECT 1 FROM information_schema.TABLE_CONSTRAINTS
               WHERE CONSTRAINT_SCHEMA = DATABASE() AND TABLE_NAME = p_tabela
                 AND CONSTRAINT_NAME = p_nome AND CONSTRAINT_TYPE = 'CHECK') THEN
        SET @ddl = CONCAT('ALTER TABLE ', p_tabela, ' DROP CHECK ', p_nome);
        PREPARE comando FROM @ddl;
        EXECUTE comando;
        DEALLOCATE PREPARE comando;
    END IF;
    SET @ddl = CONCAT('ALTER TABLE ', p_tabela, ' ADD CONSTRAINT ', p_nome, ' CHECK (',
p_expressao, ')');
    PREPARE comando FROM @ddl;
    EXECUTE comando;
    DEALLOCATE PREPARE comando;
END$$

DELIMITER ;

CALL sp_garantir_coluna('agendamentos', 'ObservacoesAtendimento', 'TEXT NULL');
CALL sp_garantir_coluna('agendamentos', 'DataChegada', 'DATETIME(6) NULL');
CALL sp_garantir_coluna('agendamentos', 'ValorPago', 'DECIMAL(10,2) NULL');
CALL sp_garantir_coluna('agendamentos', 'FormaPagamento', 'INT NULL');
CALL sp_garantir_coluna('agendamentos', 'DataPagamento', 'DATETIME(6) NULL');
CALL sp_recriar_check('funcionarios', 'CK_funcionarios_cargo', 'Cargo BETWEEN 0 AND 3');

-- ============================================================
-- 3. TRIGGERS
-- ============================================================
DROP TRIGGER IF EXISTS trg_agendamento_calcula_fim;
DROP TRIGGER IF EXISTS trg_item_venda_baixa_estoque;
DROP TRIGGER IF EXISTS trg_item_venda_atualiza_total;
DROP TRIGGER IF EXISTS trg_agendamento_log_status;
DROP TRIGGER IF EXISTS trg_estoque_resolve_alerta;

DELIMITER $$

-- RN10: o fim do agendamento é sempre o início somado à duração do serviço
CREATE TRIGGER trg_agendamento_calcula_fim
BEFORE INSERT ON agendamentos
FOR EACH ROW
BEGIN
    DECLARE v_duracao INT;
    SELECT DuracaoMinutos INTO v_duracao
    FROM servicos WHERE Id = NEW.IdServico;
    IF v_duracao IS NOT NULL THEN
        SET NEW.DataHoraFim = DATE_ADD(NEW.DataHoraInicio, INTERVAL v_duracao MINUTE);
    END IF;
END$$

-- RN15: toda venda diminui o estoque e gera alerta no ponto de reposição
CREATE TRIGGER trg_item_venda_baixa_estoque
AFTER INSERT ON itensvenda
FOR EACH ROW
BEGIN
    UPDATE estoques
    SET QuantidadeAtual = QuantidadeAtual - NEW.Quantidade,
```

```
        UltimaAtualizacao = CURRENT_TIMESTAMP(6)
    WHERE IdProduto = NEW.IdProduto;

    -- Se atingiu o ponto de reposição, dispara o alerta
    INSERT INTO alerta_reposicao (IdProduto, QuantidadeAtual, PontoReposicao)
    SELECT IdProduto, QuantidadeAtual, PontoReposicao
    FROM estoques
    WHERE IdProduto = NEW.IdProduto
      AND QuantidadeAtual <= PontoReposicao;
END$$

-- Recalcula o total da venda automaticamente quando um item é inserido
CREATE TRIGGER trg_item_venda_atualiza_total
AFTER INSERT ON itensvenda
FOR EACH ROW
BEGIN
    UPDATE vendas
    SET Total = (SELECT IFNULL(SUM(Subtotal), 0) FROM itensvenda WHERE IdVenda = NEW.IdVenda)
    WHERE Id = NEW.IdVenda;
END$$

-- RN04: registra toda mudança de status do agendamento (auditoria)
CREATE TRIGGER trg_agendamento_log_status
AFTER UPDATE ON agendamentos
FOR EACH ROW
BEGIN
    IF OLD.Status <> NEW.Status THEN
        INSERT INTO log_agendamento_status (IdAgendamento, StatusAnterior, StatusNovo)
        VALUES (NEW.Id, OLD.Status, NEW.Status);
    END IF;
END$$

-- RN15: quando a reposição devolve o saldo acima do ponto, os alertas abertos são encerrados
CREATE TRIGGER trg_estoque_resolve_alerta
AFTER UPDATE ON estoques
FOR EACH ROW
BEGIN
    IF NEW.QuantidadeAtual > NEW.PontoReposicao AND OLD.QuantidadeAtual <= OLD.PontoReposicao
THEN
        UPDATE alerta_reposicao
        SET Resolvido = 1
        WHERE IdProduto = NEW.IdProduto AND Resolvido = 0;
    END IF;
END$$

DELIMITER ;

-- ============================================================
-- 4. PROCEDURES
-- ============================================================
DROP PROCEDURE IF EXISTS sp_agendar_servico;
DROP PROCEDURE IF EXISTS sp_horarios_disponiveis;
DROP PROCEDURE IF EXISTS sp_registrar_venda;
DROP PROCEDURE IF EXISTS sp_receita_mensal;
DROP PROCEDURE IF EXISTS sp_estoque_critico;

DELIMITER $$

-- Procedure 1: agenda um serviço validando data futura, expediente (RN03) e conflito (RN01)
CREATE PROCEDURE sp_agendar_servico(
    IN p_id_pet INT,
    IN p_id_servico INT,
    IN p_id_funcionario INT,
    IN p_data_hora_inicio DATETIME,
    IN p_observacoes TEXT,
    OUT p_id_agendamento INT,
    OUT p_mensagem VARCHAR(255)
)
BEGIN
    DECLARE v_duracao INT;
    DECLARE v_data_hora_fim DATETIME;
    DECLARE v_conflito INT DEFAULT 0;

    SET p_id_agendamento = 0;

    -- RN10: duração do serviço
```

```
    SELECT DuracaoMinutos INTO v_duracao
    FROM servicos WHERE Id = p_id_servico AND Ativo = 1;

    SET v_data_hora_fim = DATE_ADD(p_data_hora_inicio, INTERVAL v_duracao MINUTE);

    IF v_duracao IS NULL THEN
        SET p_mensagem = 'Serviço não encontrado.';
    ELSEIF p_data_hora_inicio <= NOW() THEN
        SET p_mensagem = 'A data do agendamento deve ser futura.';
    ELSEIF DAYOFWEEK(p_data_hora_inicio) = 1 THEN          -- 1 = domingo
        SET p_mensagem = 'Fechado aos domingos (RN03).';
    ELSEIF TIME(p_data_hora_inicio) < '08:00:00'
        OR TIME(v_data_hora_fim) > '19:00:00'
        OR DATE(v_data_hora_fim) <> DATE(p_data_hora_inicio) THEN
        SET p_mensagem = 'Fora do horário de funcionamento, das 08h às 19h (RN03).';
    ELSE
        -- RN01: verifica sobreposição com o mesmo profissional
        SELECT COUNT(*) INTO v_conflito
        FROM agendamentos
        WHERE IdFuncionario = p_id_funcionario
          AND Status <> 3                                  -- ignora os cancelados
          AND p_data_hora_inicio < DataHoraFim
          AND v_data_hora_fim > DataHoraInicio;

        IF v_conflito > 0 THEN
            SET p_mensagem = 'Conflito de horário com outro atendimento (RN01).';
        ELSE
            INSERT INTO agendamentos
                (IdPet, IdServico, IdFuncionario, DataHoraInicio, DataHoraFim, Status,
Observacoes, DataCriacao)
            VALUES
                (p_id_pet, p_id_servico, p_id_funcionario, p_data_hora_inicio,
v_data_hora_fim, 0, p_observacoes, CURRENT_TIMESTAMP(6));
            SET p_id_agendamento = LAST_INSERT_ID();
            SET p_mensagem = 'Agendamento criado com sucesso.';
        END IF;
    END IF;
END$$

-- Procedure 2: retorna os horários livres de um profissional em uma data (intervalos de 15
minutos)
CREATE PROCEDURE sp_horarios_disponiveis(
    IN p_data DATE,
    IN p_id_servico INT,
    IN p_id_funcionario INT
)
BEGIN
    DECLARE v_duracao INT;
    DECLARE v_inicio DATETIME;
    DECLARE v_fechamento DATETIME;

    SELECT DuracaoMinutos INTO v_duracao FROM servicos WHERE Id = p_id_servico;

    DROP TEMPORARY TABLE IF EXISTS horarios_disponiveis_temp;
    CREATE TEMPORARY TABLE horarios_disponiveis_temp (data_hora DATETIME);

    SET v_inicio = TIMESTAMP(p_data, '08:00:00');          -- Abertura (RN03: das 08h às 19h)
    SET v_fechamento = TIMESTAMP(p_data, '19:00:00');

    -- Sem expediente aos domingos; o atendimento precisa terminar até o fechamento
    WHILE v_duracao IS NOT NULL
      AND DAYOFWEEK(p_data) <> 1
      AND DATE_ADD(v_inicio, INTERVAL v_duracao MINUTE) <= v_fechamento DO
        IF v_inicio > NOW() AND NOT EXISTS (
            SELECT 1 FROM agendamentos
            WHERE IdFuncionario = p_id_funcionario
              AND Status <> 3
              AND v_inicio < DataHoraFim
              AND DATE_ADD(v_inicio, INTERVAL v_duracao MINUTE) > DataHoraInicio
        ) THEN
            INSERT INTO horarios_disponiveis_temp (data_hora) VALUES (v_inicio);
        END IF;
        SET v_inicio = DATE_ADD(v_inicio, INTERVAL 15 MINUTE);
    END WHILE;

    SELECT data_hora FROM horarios_disponiveis_temp ORDER BY data_hora;
```

```
    DROP TEMPORARY TABLE IF EXISTS horarios_disponiveis_temp;
END$$

-- Procedure 3: registra uma venda completa em uma única transação.
-- p_itens é um JSON no formato [{"idProduto": 1, "quantidade": 2}, ...].
-- A baixa de estoque e o total são feitos pelas triggers de itensvenda; se faltar
-- saldo para qualquer item, a venda inteira é desfeita (ROLLBACK).
CREATE PROCEDURE sp_registrar_venda(
    IN p_id_tutor INT,
    IN p_id_funcionario INT,
    IN p_forma_pagamento INT,
    IN p_observacoes TEXT,
    IN p_itens JSON,
    OUT p_id_venda INT,
    OUT p_mensagem VARCHAR(255)
)
BEGIN
    DECLARE v_sem_saldo INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_id_venda = 0;
        SET p_mensagem = 'Venda não registrada: dados inválidos ou estoque insuficiente.';
    END;

    SET p_id_venda = 0;

    START TRANSACTION;

    -- Itens pedidos acima do saldo disponível (ou sem registro de estoque)
    SELECT COUNT(*) INTO v_sem_saldo
    FROM JSON_TABLE(p_itens, '$[*]' COLUMNS (
            idProduto INT PATH '$.idProduto',
            quantidade INT PATH '$.quantidade')) AS i
    LEFT JOIN estoques e ON e.IdProduto = i.idProduto
    WHERE e.Id IS NULL OR e.QuantidadeAtual < i.quantidade OR i.quantidade <= 0;

    IF v_sem_saldo > 0 OR JSON_LENGTH(p_itens) = 0 THEN
        ROLLBACK;
        SET p_mensagem = 'Venda não registrada: estoque insuficiente para um ou mais itens.';
    ELSE
        INSERT INTO vendas (IdTutor, IdFuncionario, FormaPagamento, Observacoes, Total,
DataVenda)
        VALUES (p_id_tutor, p_id_funcionario, p_forma_pagamento, p_observacoes, 0,
CURRENT_TIMESTAMP(6));
        SET p_id_venda = LAST_INSERT_ID();

        INSERT INTO itensvenda (IdVenda, IdProduto, Quantidade, PrecoUnitario, Subtotal)
        SELECT p_id_venda, p.Id, i.quantidade, p.PrecoVenda, p.PrecoVenda * i.quantidade
        FROM JSON_TABLE(p_itens, '$[*]' COLUMNS (
                idProduto INT PATH '$.idProduto',
                quantidade INT PATH '$.quantidade')) AS i
        INNER JOIN produtos p ON p.Id = i.idProduto;

        COMMIT;
        SET p_mensagem = 'Venda registrada com sucesso.';
    END IF;
END$$

-- Procedure 4: relatório de receita mensal (produtos vendidos + serviços concluídos)
CREATE PROCEDURE sp_receita_mensal(
    IN p_ano INT,
    IN p_mes INT
)
BEGIN
    DECLARE v_inicio DATETIME DEFAULT STR_TO_DATE(CONCAT(p_ano, '-', p_mes, '-01'), '%Y-%m-
%d');
    DECLARE v_fim DATETIME DEFAULT DATE_ADD(v_inicio, INTERVAL 1 MONTH);

    SELECT
        DATE_FORMAT(v_inicio, '%Y-%m') AS competencia,
        (SELECT COUNT(*) FROM vendas WHERE DataVenda >= v_inicio AND DataVenda < v_fim) AS
total_vendas,
        (SELECT IFNULL(SUM(Total), 0) FROM vendas WHERE DataVenda >= v_inicio AND DataVenda <
v_fim) AS receita_produtos,
```

```
        (SELECT IFNULL(AVG(Total), 0) FROM vendas WHERE DataVenda >= v_inicio AND DataVenda <
v_fim) AS ticket_medio,
        (SELECT IFNULL(SUM(IFNULL(a.ValorPago, s.Preco)), 0)   -- valor recebido no balcão, ou
o preço de tabela
           FROM agendamentos a INNER JOIN servicos s ON s.Id = a.IdServico
          WHERE a.Status = 2 AND a.DataHoraInicio >= v_inicio AND a.DataHoraInicio < v_fim) AS
receita_servicos;
END$$


-- Procedure 5: produtos em estoque crítico (ponto de reposição)
CREATE PROCEDURE sp_estoque_critico()
BEGIN
    SELECT Id, Nome, QuantidadeAtual, PontoReposicao, SaldoAbaixoPonto
    FROM vw_estoque_critico
    ORDER BY QuantidadeAtual ASC;
END$$


DELIMITER ;


-- ============================================================
-- 5. VIEWS
-- ============================================================


-- Visão de estoque crítico para alertas (RN15)
CREATE OR REPLACE VIEW vw_estoque_critico AS
SELECT
    p.Id,
    p.Nome,
    e.QuantidadeAtual,
    e.PontoReposicao,
    (e.QuantidadeAtual - e.PontoReposicao) AS SaldoAbaixoPonto
FROM estoques e
INNER JOIN produtos p ON p.Id = e.IdProduto
WHERE e.QuantidadeAtual <= e.PontoReposicao;

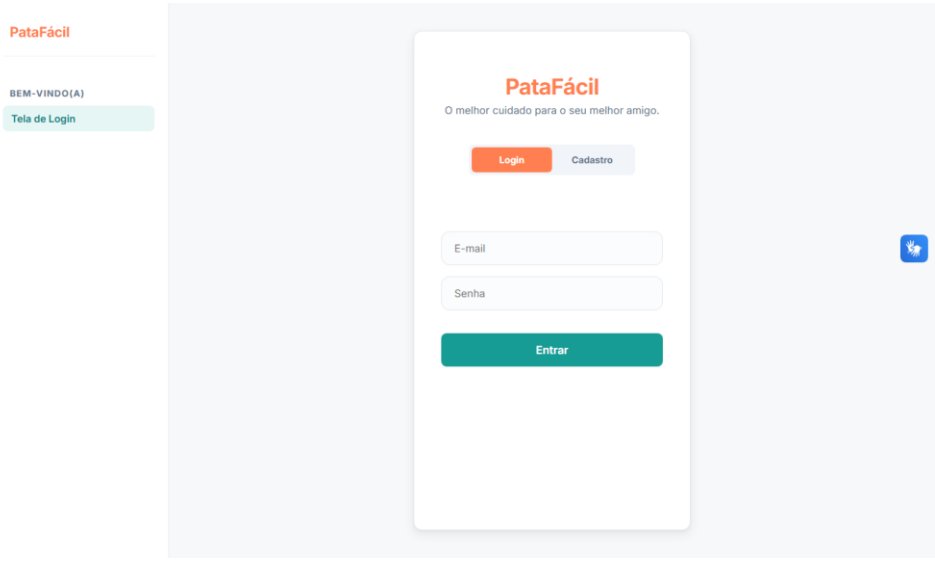

-- Visão de receita por serviço (apoia relatórios gerenciais)
CREATE OR REPLACE VIEW vw_receita_por_servico AS
SELECT
    s.Id,
    s.Nome AS Servico,
    COUNT(a.Id) AS QtdAtendimentos,
    IFNULL(SUM(IF(a.Id IS NULL, 0, s.Preco)), 0) AS ReceitaEstimada
FROM servicos s
LEFT JOIN agendamentos a ON a.IdServico = s.Id AND a.Status IN (1, 2) -- CONFIRMADO/CONCLUIDO
GROUP BY s.Id, s.Nome;


-- Visão da agenda com os nomes resolvidos (pet, tutor, serviço e profissional)
CREATE OR REPLACE VIEW vw_agenda_detalhada AS
SELECT
    a.Id,
    a.DataHoraInicio,
    a.DataHoraFim,
    a.Status,
    p.Nome AS Pet,
    t.Nome AS Tutor,
    s.Nome AS Servico,
    f.Nome AS Profissional
FROM agendamentos a
INNER JOIN pets p ON p.Id = a.IdPet
INNER JOIN tutores t ON t.Id = p.TutorId
INNER JOIN servicos s ON s.Id = a.IdServico
INNER JOIN funcionarios f ON f.Id = a.IdFuncionario;
```

APÊNDICE B – TELAS COMPLEMENTARES DA APLICAÇÃO WEB

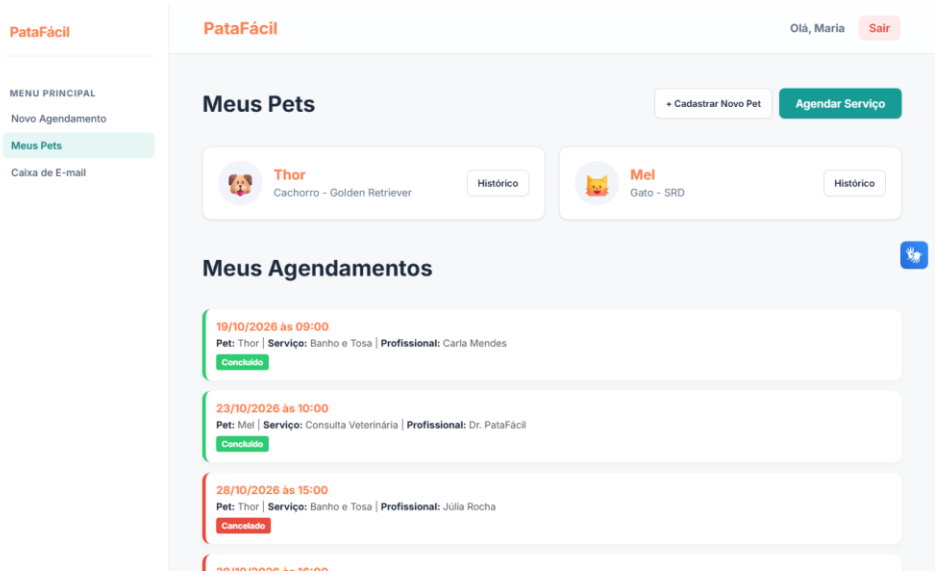
As figuras a seguir complementam a seção 5.5 e foram capturadas com a aplicação em execução.

Figura 16 – Tela de login e cadastro



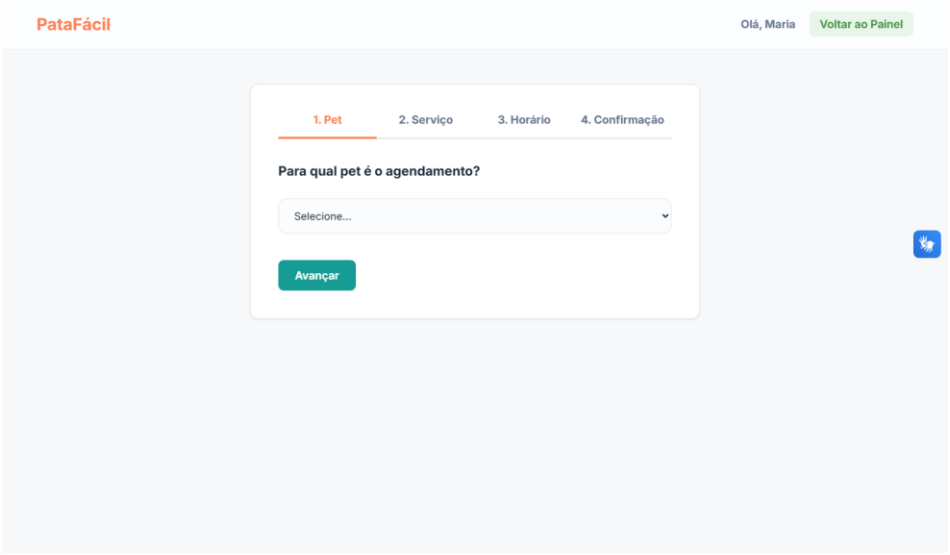
Fonte: elaborado pelo autor (2026).

Figura 17 – Portal do tutor



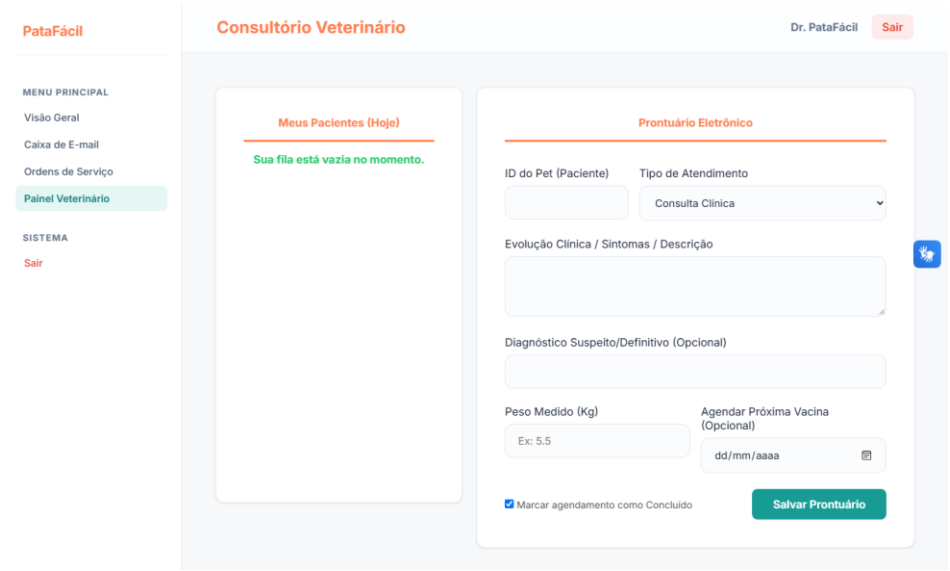
Fonte: elaborado pelo autor (2026).

Figura 18 – Assistente de agendamento da aplicação web



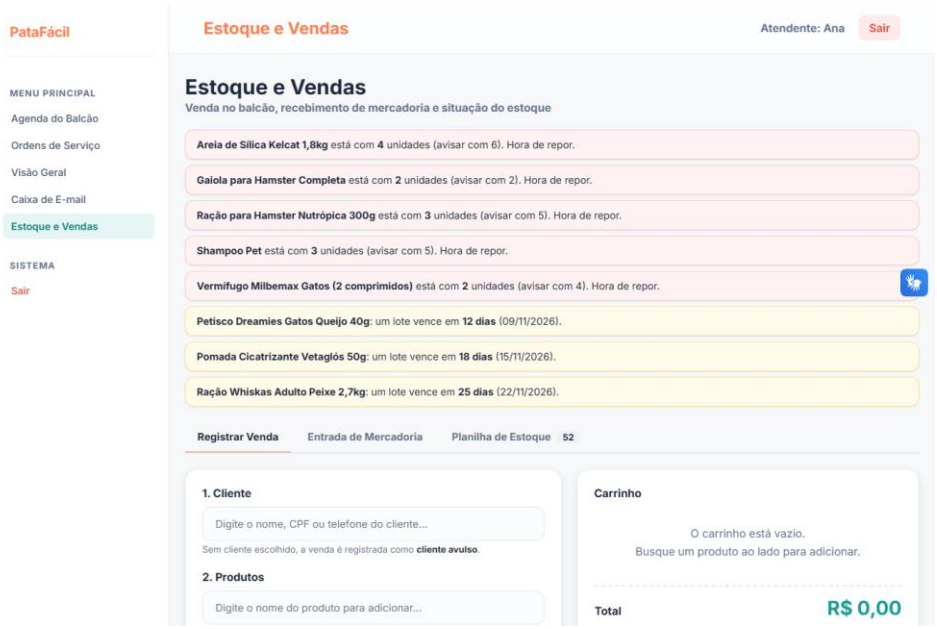
Fonte: elaborado pelo autor (2026).

Figura 19 – Consultório veterinário e prontuário eletrônico



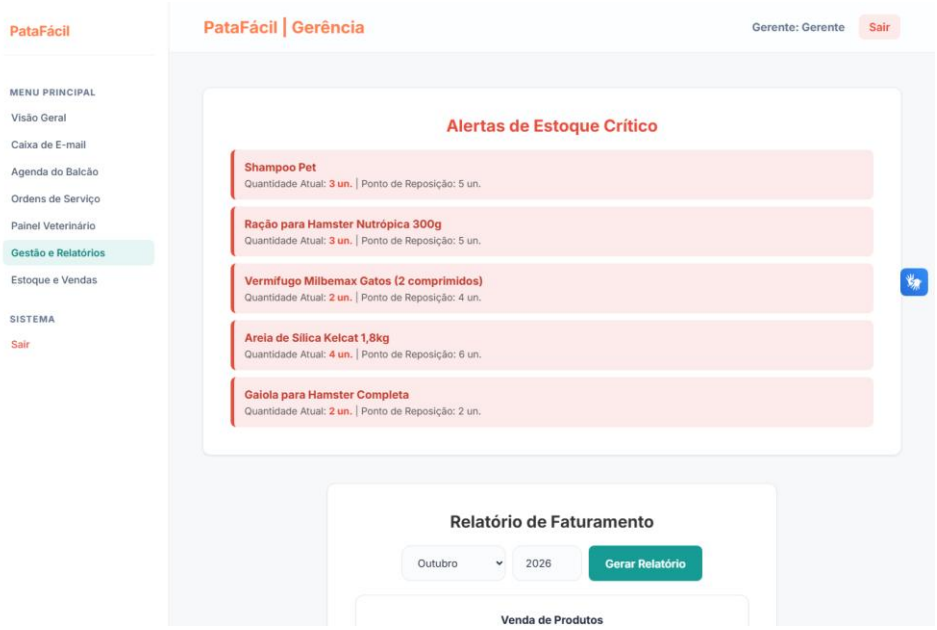
Fonte: elaborado pelo autor (2026).

Figura 20 – Venda no balcão com carrinho e alertas de estoque



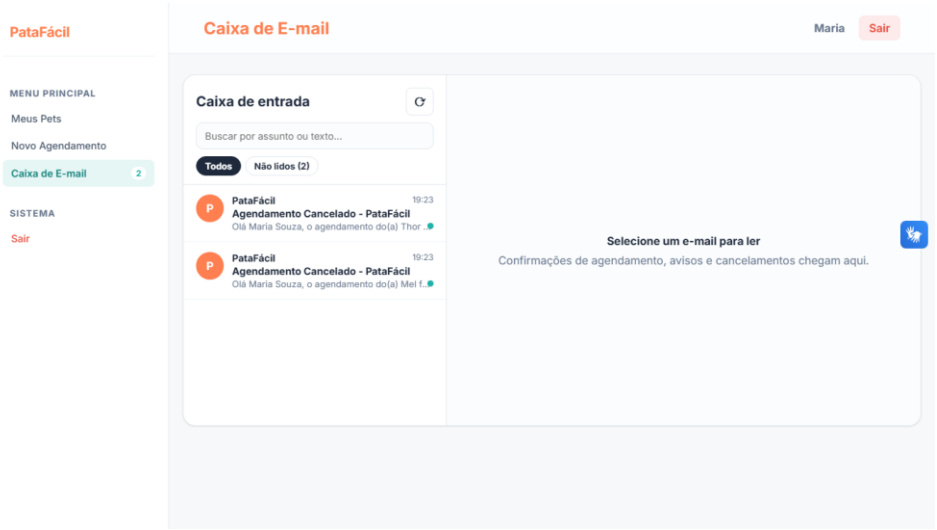
Fonte: elaborado pelo autor (2026).

Figura 21 – Painel do gerente com o relatório de faturamento



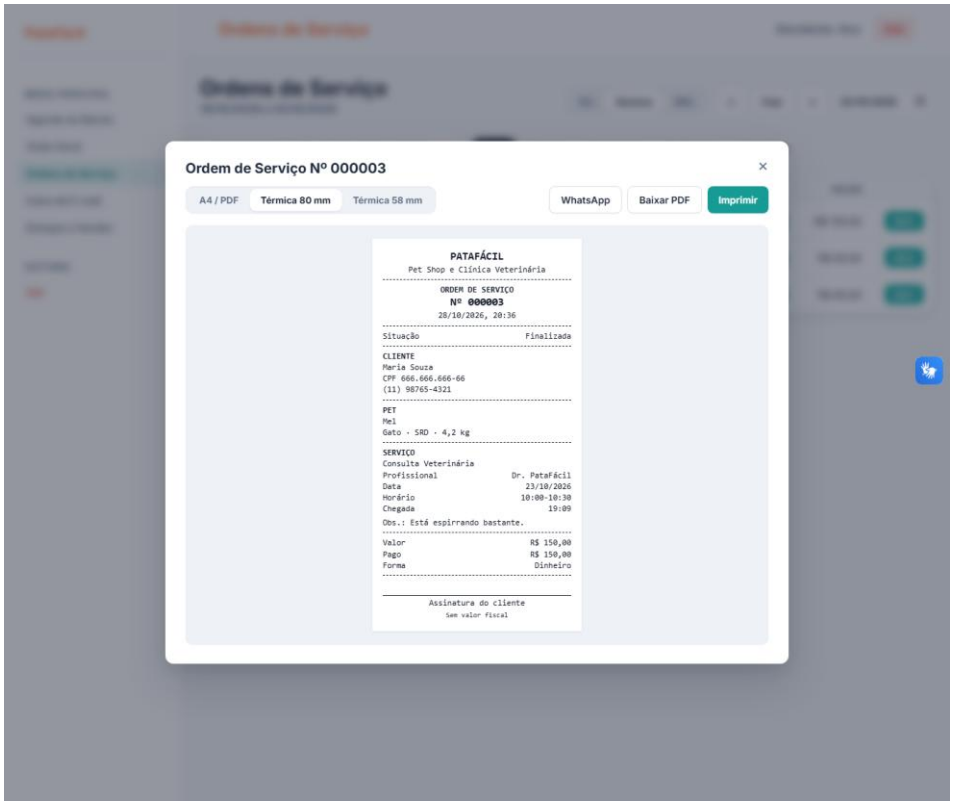
Fonte: elaborado pelo autor (2026).

Figura 22 – Caixa de e-mail do usuário



Fonte: elaborado pelo autor (2026).

Figura 23 – Ordem de serviço no formato de cupom para impressora térmica de 80 mm



Fonte: elaborado pelo autor (2026).